

Onyx — Developer Guide

Onyx · multi-process OS on Raspberry Pi 4

This guide explains how to **build** Onyx, how to **write an application** or a **/bin tool**, how to **extend the kapi ABI**, and the **conventions** + **pitfalls** to be aware of. For the details of how things work internally, see [Kernel internals](#).

Contents

1. Prerequisites and toolchain
 2. Building Circle (once)
 3. Building the kernel and applications
 4. Deploying to the SD card
 5. The application model
 6. Writing a graphical application
 7. Writing a /bin tool
 8. The `applib.h` library
 9. Packaging an app: `.app`, icons, `config.ini`
 10. Extending the `kapi` ABI
 11. Coding conventions
 12. Debugging on hardware
 13. Known pitfalls
-

1. Prerequisites and toolchain

- **AArch64 bare-metal toolchain:** `aarch64-none-elf-` (GCC), used under **WSL** on a Windows development machine.
- **Circle**, pulled in as a **git submodule** at `circle/` (our fork `stephaneweg/circle`, branch `onyx`).
- `make`, `cp`, `mkdir` (standard Unix tools, via WSL).
- A **FAT32 SD card** and a **Raspberry Pi 4** to run on (there is no QEMU `raspi4b` in the reference dev environment; bring-up is done directly on hardware).

Important — which Circle? `circle/` is a **git submodule** → our fork `stephaneweg/circle` (branch `onyx`), **not** any other clone. Patch Circle *there* (commit on `onyx`), rebuild the affected library, then record the new commit in the superproject (`git add circle && git commit`). The patches are listed in [Circle Changes](#).

2. Building Circle (once)

```
# Circle is a git submodule (the fork stephaneweg/circle, branch onyx). Fetch it
# (or clone Onyx with --recurse-submodules in the first place):
git submodule update --init --recursive
```

```
# Configure for RPi 4 / AArch64. DEPTH=32 is REQUIRED: our GImage renders 32-bit
# pixels (Circle defaults to 16).
cd circle && ./configure -r 4 -p aarch64-none-elf- -d DEPTH=32 -f
```

```
# Build the libraries used by the kernel (make clean first when the fork's sysconfig.h
# changed -- e.g. the multi-core patch: the .o files do not depend on it):
for d in lib lib/sched lib/fs lib/fs/fat lib/usb lib/input lib/net lib/sound \
    addon/SDCard addon/fatfs addon/wlan; do (cd $d && make clean && make -j4) || break;
done
(cd addon/wlan/hostap/wpa_supplicant && make -f Makefile.circle clean && make -f
Makefile.circle -j4)
cd ..
```

If you change DEPTH later, run `make clean` in `circle/lib` before rebuilding (the `.o` files do not depend on `Config.mk`).

The libraries linked by the kernel (cf. `kernel/Makefile`): `libwpa_supplicant.a`, `libwlan.a`, `libfatfs.a`, `libsdcard.a`, `libnet.a`, `libsound.a`, `libsched.a`, `libusb.a`, `libinput.a`, `libfs.a`, `libcircle.a`. Circle is built **multi-core** (`ARM_ALLOW_MULTI_CORE`, fork patch #4): core 1 runs the sound producer.

The Onyx-specific patches carried by this fork (branch `onyx`, on upstream tag `Step51`) are documented in [Circle Changes](#).

3. Building the kernel and applications

The kernel's size limit. The kernel image **and its BSS** must end below `0x280000` (loaded at `0x80000`, Circle's `KERNEL_MAX_SIZE` = 2 MB): past it the BSS runs over the kernel's stacks and the Pi does not boot at all, without a message. `make` checks it after each link (`sizecheck: _end` in `kernel8-rpi4.map`) and deletes an image too big. Put big buffers on the heap (`new` at `init`), not in static arrays.

From `kernel/`:

```
cd kernel
make          # -> kernel8-rpi4.img, THEN builds the apps (../user)
```

The default target (`all`):

1. compiles our sources + links against the Circle libs → `kernel8-rpi4.img`;
2. triggers `make -C ../user`, which builds **all the apps** (`user/*.elf`) and the `/bin tools` (`user/bin/*.elf`).

Kernel → apps order. Since the apps go through the **fixed-address ABI table** and are **not** linked against the kernel's addresses, they no longer depend on the kernel build: a *kernel-only* change (that respects the ABI) does **not** require recompiling the apps. `make` rebuilds them anyway for convenience.

Details of the kernel wiring (which Circle files we replace, the order of the `-I compat` `-I include` include path before `circle/include`): see [Kernel internals §1](#) and `kernel/Makefile`.

4. Deploying to the SD card

```
cd kernel
make stage      # copie l'image + chaque app + chaque outil vers ../sdcard/
```

`make stage`:

- copies `kernel8-rpi4.img` → `sdcard/`;
- for each `../user/<name>.elf`: creates `sdcard/apps/<name>.app/` and copies the ELF into it as `main` (the "Onyx" layout);

- copies each `../user/bin/<tool>.elf` → `sdcard/bin/<tool>`.

Executables on the card have no extension: `.elf` only exists in the build tree (`user/*.elf`, `user/bin/*.elf`); staging drops it. Programs are recognized by their content (ELF magic `7F 45 4C 46`), e.g. by the file manager.

Then copy **all** of the contents of `sdcard/` onto a **FAT32** card and boot the Pi 4. See the [user guide](#) for details about the card and the configuration files.

Keyboard layouts live as binary files in `sdcard/etc/keymaps/<NAME>.kmap` (the `keyb` tool — and the theme editor's dropdown, which lists whatever `.kmap` files are actually present — load one and send it to the kernel via `kapi_set_keymap_data`, v27). **The kernel compiles in no keyboard map at all:** at boot the keyboard table is empty and stays so until `keyb` loads a layout from the card (see [Circle Changes §1](#)), so a layout is *only* ever a file — adding one needs no kernel rebuild. Regenerate the `.kmap` files with `python tools/keymaps/genkeymaps.py`; it compiles each `<NAME>` from a `keymap_<name>.h` table in `tools/keymaps/maps/` (the 8 layout sources, incl. `BE` = Belgian azerty, all tracked in the Onyx repo — not in the Circle tree). The `.kmap` format is `"OKM1"` + `u16` rows/cols + the `u16[128][5]` table (see the script header).

5. The application model

A Onyx application is a **single .c file** compiled into a **freestanding ELF** and run in **EL1** in its own page table.

Runtime (`user/crt0.S`):

```
_start:
    stp x29, x30, [sp, #-16]!    /* the stack is already set up by the kernel */
    bl  main                    /* call main() */
    ldp x29, x30, [sp], #16
    ret                         /* return -> the kernel terminates the task */
```

No `argv` is passed via the stack: `main()` takes no arguments. An app retrieves its argument line via `kapi_get_args(buf, size)`, and exits early with `kapi_exit(status)`.

Linking (`user/user.ld`): everything is linked at **USER_VA_BASE = 8 GB** (`. = 0x200000000`;). This is **not PIE**. The `RX` (`.text/.rodata`) and `RW` (`.data/.bss`) sections are aligned on **64 KB** (`-z max-page-size=0x10000`) so that the loader maps them onto distinct pages. **No kernel symbol** is resolved at link time: everything goes through the ABI table.

Compilation flags (`user/Makefile`):

```
-ffreestanding -nostdlib -fno-pic -fno-pie -mgeneral-regs-only \
-O2 -Wall -Wextra -fno-stack-protector -I../kernel/include
```

- `-ffreestanding -nostdlib`: **no libc**. No `printf`, `malloc`, `string.h`... use `applib.h` (§8) and static/local buffers.
- `-mgeneral-regs-only`: integer-only codegen — the **default** for apps. Most apps do **integer / fixed-point arithmetic** (see [tinycalc](#), [mandelbrot](#)).
- **Hardware float is available (opt-in)**. The kernel now saves the full FP/SIMD state on every trap (see [Kernel internals §6](#), trap frame), and FP/SIMD is enabled at EL0 (`CPACR_EL1`). To use `float/double` in an app, build it **without** `-mgeneral-regs-only` and **with** `-mcpu=cortex-a72` (the Pi 4 core). Basic FP (`+ - * /`, `int↔double`) compiles to hardware instructions with **no** soft-float

runtime; transcendental math (`sin/cos/ pow...`) lives in newlib's `libm` — see §5.1. For the bare-FP recipe, see the `fpctest` target in `user/bin/Makefile` (a target-specific `CFLAGS` override) and the `fpctest` proof tool. Opt **only** the FP app in — leave the rest integer-only.

- `-I../kernel/include`: to include `<kern/kapi_abi.h>` (the shared ABI structure).

To add an app, create `user/<name>.c` and **add `<name>.elf` to the `PROGS` variable** of `user/Makefile` (and `<tool>.elf` to the `PROGS` of `user/bin/Makefile` for a tool).

5.1. Apps using the C library (newlib)

The `aarch64-none-elf` toolchain ships **newlib** (`libc` + `libm`). An app can be built against it to use the real `<stdio.h>` (`printf`, `FILE*`, `fopen/fseek`), `<stdlib.h>` (`malloc/qsort/strtod`), `<string.h>`, and `<math.h>` instead of the freestanding helpers (`applib.h/umm.h`). This is the foundation for porting large C codebases (e.g. NetSurf).

How it works: `user/libc/onyx_syscalls.c` implements the handful of POSIX stubs newlib bottoms out in (`_sbrk`, `_read`, `_write`, `_open`, `_close`, `_lseek`, `_fstat`, `_gettimeofday`, ...) on top of the kapi ABI. `user/libc/crt0libc.S` is the entry point — like `crt0.S` but it calls `exit(main())` so `stdio` is flushed on the way out. Build flags drop `-ffreestanding/-nostdlib` and use `-nostartfiles` (keep our entry, keep `libc`); add `-mcpu=cortex-a72` (FP is required by `printf %f` and `libm`) and link `-lm`. See the `LIBC_PROGS` rule in `user/bin/Makefile` and the proof tool `user/bin/libctest.c`.

A **wtk app** can be a newlib app too (Doom, NetSurf, **Writer**, the **Spreadsheet** — FreeType wants a `libc`): the `writer.elf` rule of `user/Makefile` is the model — `NL_CFLAGS` / `NL_CXXFLAGS` (hardware FP, `-nostartfiles`, sections for `--gc-sections`), `libc/crt0libc.o` + `libc/onyx_syscalls.o`, the app, `wtk/libwtk.a`, then its libraries (`ft/libft.a`) and `-lm`. Take the app out of the generic `APPS` list and add its `.elf` to `all`; `make stage` stages it as any.

Notes / caveats:

- **One allocator** for a plain C newlib app: newlib's `malloc` owns the heap via `_sbrk`→`kapi_sbrk`; do **not** also link `umm.h`. (A wtk app on newlib has two, side by side: wtk's C++ objects on `umm` — `onyxpp.hpp`'s operator `new` — and the C libraries on `malloc`; each grows its own arena with `kapi_sbrk`.)
- **Files are buffered in RAM**. The kapi file API is sequential (no seek), so `_open` slurps the whole file into memory to give `fseek/fte11` full semantics, and a writable file is written back with `kapi_save_file()` on `close`. Fine for resource files; a future `kapi_lseek` would remove the whole-file buffering.
- **Size**. Static newlib pulls a fair amount of code (`printf` float support etc.). Acceptable for big apps; a `nano` variant can be revisited later for small tools.
- **Threads** (§5.2): newlib's locks are defined in `onyx_syscalls.c` (newlib is built with retargetable locking), so `malloc`, the `FILES`, `atexit...` are safe between threads. `errno` and newlib's other reentrancy state (`_impure_ptr`) are **shared** by the threads.

5.2. Threads (kapi v67)

A process may run **threads**: more tasks in its own address space — the same memory, window, files and sockets —, preempted like the main one, all on core 0. What they are for is **concurrency**: a thread blocks (a socket, a file, a long computation) while the main thread keeps pumping the window's events. (For parallel computing on another core, see the app cores, §6.)

```

static int worker (void *arg)          /* runs in the new thread */
{
    long n = fetch_something (arg);    /* blocks: the UI goes on meanwhile */
    kapi_post (on_done, arg, n);       /* on_done (arg, n) runs on the main thread */
    return 0;                          /* its exit code */
}

int tid = kapi_thread_create (worker, ctx, 0, "fetch"); /* 0: a 256 KB stack */
...
while (!kapi_should_exit ())
    kapi_pump_wait (100);              /* sleeps until an event, a post or the close box */
*/
kapi_thread_join (tid, KAPI_WAIT_FOREVER, &code);

```

- **Threads:** `kapi_thread_create (fn, arg, stack_size, name)` → a tid ≥ 2 (the main thread is 1), -1 no memory, -2 too many (**32** besides the main one). The stack is 256 KB by default, 16 KB .. 16 MB. The name is <app>:<name> (in the crash reports; `ps`, the task manager and the taskbar show the app once). `fn`'s return value — or `kapi_thread_exit (code)` — is the exit code, which `kapi_thread_join (tid, timeout_ms, &code)` collects (-1 timeout, -2 no such thread or joined already, -3 itself); a thread never joined is forgotten once its record is needed again. `kapi_thread_self ()` → the caller's tid.
- **The process ends with its main thread** (returning from `main`, `kapi_exit`), or when any thread calls `kapi_exit`: the other threads end with it, wherever they are. Killing the app (the task manager, `kill`) kills them all.
- **Synchronisation objects** — handles, 256 per process, freed with it or by `kapi_sync_close (h)` (its waiters get -2). Timeouts are in ms: 0 only tries, `KAPI_WAIT_FOREVER` waits.
 - **mutex:** `kapi_mutex_create (), kapi_mutex_lock (h, timeout) (0, -1 timeout), kapi_mutex_unlock (h)`. Recursive; released if its owner thread ends.
 - **event:** `kapi_event_create (manual_reset, initial), kapi_event_set, kapi_event_reset, kapi_event_wait (h, timeout)`. A manual-reset event stays set (it releases every waiter); an auto-reset one is taken back by the wait that gets it (one waiter per set).
 - **barrier:** `kapi_barrier_create (count), kapi_barrier_wait (h)` — returns once `count` threads are in: 1 for the last one in, 0 for the others; the barrier is then ready for the next round.
- **Calls posted to the pump:** `kapi_post (fn, ctx, value)` queues `fn (ctx, value)` (256 at most: -1 when full); the pump runs it — `kapi_pump_events, kapi_pump_wait, kapi_wait_for_exit` — on the thread that pumps. That is how a worker hands its result to the UI: no lock needed on the UI's data. `kapi_pump_wait (ms)` sleeps until a window event, a post or the close box (or the timeout), then pumps → what was pending (0: the timeout). `kapi_wait_for_exit` wakes on a post too.
- **Rules:**
 - **The GUI belongs to the thread that pumps** (the main one): draw, present, open dialogs and change the window from it; other threads post to it.
 - **Memory is shared:** protect what several threads write (a mutex, or post the work to one thread). `umm_malloc / umm_free` and newlib's `malloc` take a lock; `errno` is shared.

- For a small lock of your own there is `kapi_lock (&int) / kapi_unlock (&int)` (a swap and a yield: no handle, a zeroed `int` is a free lock; not recursive).
 - A kapi call is not preempted, but it may wait (a file read in pieces, a socket): another thread of the process can run meanwhile, even in another kapi call.
- Example and test: `user/bin/threadtest.c` (`threadtest` in a terminal: every check, then PASS / FAIL).
- **Word waits — a futex (v68):** `kapi_wait_word (&word, expected, timeout_ms)` sleeps while `word == expected` → 0 (woken, or the value already differs), 1 timeout (0 ms: only check), -1 a bad address (not 4-byte aligned, unmapped). `kapi_wake_word (&word)` wakes its sleepers → how many. Nothing to create: any `unsigned` of your memory, your stack or a **shared surface** — the kernel keys the sleepers by the physical address, so two processes that map the same surface (a plugin and its host) wait / wake on the same word. **Code on an app core** changes words without calling anything (it cannot): the kernel reads every sleeping word at each 10 ms tick and wakes those that changed — so a core-0 thread can sleep until the engine on core 2 has written (≤ 10 ms late; call `kapi_wake_word` when you can, it is immediate). Wakes may be spurious: loop on your condition.

```
while (ring->ready == 0) /* set by another thread / process /
core */
    kapi_wait_word (&ring->ready, 0, 100);
```

- **Priority (v68):** `kapi_thread_priority (tid, 1)` (tid 0 = the caller, 1 = the main thread) makes a thread "**real time**": whenever it is ready it runs before the others, and the next tick preempts an app for it. It keeps that only while it sleeps / waits before its 20 ms slice ends — a thread that computes through its slice is an ordinary one until it next sleeps, so it cannot freeze the system. For an audio pump, a plugin's render loop. -1 asks → the previous priority; -2 no such thread. Test: `user/bin/futextest.c`.

6. Writing a graphical application

Notifications and clipboard (ABI v40). `#include "notify.h"` then `notify ("My App", "Done.")` shows a bubble (the `notifyd` service, reached by IPC; launched on demand). `#include "clipboard.h"`: `clip_set_text`, `clip_get_text`, `clip_set_files (path, cut)`, `clip_get_file` — the kernel keeps one shared clipboard. **Full-screen apps (ABI v41):** `unsigned *fb = kapi_fullscreen_begin (&w, &h);` gives a screen-sized buffer; draw into it and call `kapi_present_fb ()` once per frame (it also yields). The desktop is not drawn meanwhile and all input comes to your key/pointer handlers in screen coordinates. `kapi_fullscreen_end ()` (or exiting) restores the desktop. See `user/Apps/plasma`. **v55:** `unsigned *scr = kapi_fullscreen_direct (&w, &h, &stride);` (after `fullscreen_begin`) gives the **displayed framebuffer itself** (uncached; `stride` in pixels): draw there — or render there with `kapi_gpu_render` — and nothing is copied any more (`kapi_present_fb` only yields); tearing is possible. 0: keep the back buffer. Used by `n64emu` in full screen.

Files: `fsutil.h` (`fs_join`, `fs_exists`, `fs_is_dir`, `fs_copy_tree`, `fs_remove_tree`, `fs_unique_name`) and `trash.h` (`trash_move`, `trash_restore`, `trash_purge`, `trash_empty`, `trash_count` — layout `SD:/Trash/files + info/*.trashinfo` holding `Path=<original>`). Your own IPC service: `kapi_ipc_register ("name")`, clients `kapi_ipc_lookup ("name")` + `kapi_mailbox_send (pid, type, data, len)` (≤ 512 bytes); the service drains with `kapi_mailbox_recv`.

Drag & drop (ABI v42). A **source** calls `kapi_drag_begin (DND_FILES, paths, len, label)` while the left button is held (from `onMouse`, once the cursor moved a few pixels from the press); `paths` is `\n`-separated. A **target** overrides the `wtk::Root` virtuals: `onDrop (x, y, type, data, len, flags)` (data NUL-terminated; `flags & DND_F_COPY = Ctrl` held), `onDragOver (x, y, leave, flags)` (highlight the drop spot), and the source gets `onDragDone (targetPid, flags)` (`DND_F_DESKTOP =` dropped on the desktop, `DND_F_CANCEL = Esc`). `kapi_get_modifiers ()` returns `MOD_CTRL / MOD_SHIFT / MOD_ALT`. Examples: `fileviewer` (source + target), `shelf`, and the document apps.

File associations: `#include "fileassoc.h"` — `fa_open (path)` opens a path like a double-click (folder → File Viewer, `.app / ELF` → run, else the app `SD:/etc/fileassoc.ini` maps its extension to, as `SD:apps/<app>.app/main <path>`); `fa_app_for (path, app, cap)` only looks it up. **Unsaved changes:** `#include "docguard.h"` — keep `doc_hash (data, len)` of the document as loaded / saved, and before replacing it call `doc_confirm (name, changed, save_fn)` (a Yes / No / Cancel `MB_YESNOCANCEL` box; false = cancelled). A document app should accept a path argument (`kapi_get_args`) and a dropped file (`onDrop`), so it works with the File Viewer, the Shelf and `fileassoc.ini`. **Images:** `#include "img/imgload.hpp"` (in one TU) — `img_load (path, &frames)` decodes BMP / GIF (all frames + delays) / PNG / JPEG (stb_image, public domain), WebP (simplewebp, BSD-3) and PCX (our decoder) into `0xAARRGGBB` frames from the app's umm heap (`new unsigned[] frames`); `img_free (&frames)`; `img_is_image_name (name)`. The codecs are compiled once into `libwtk.a` (`wtk/imgload.cpp`, with FP/SIMD like `wtk/canvas.o`) and linked only into the apps that call them. Users: `imageview`, `fileviewer` (preview), `wtk::ImageBox`. `img_load_mem (data, len, &frames)` decodes a file's bytes already in memory (Writer's RTF pictures, Paint's OpenRaster layers); `img_inflate (data, len, zlib, &n)` inflates a deflate stream (stb's: a ZIP entry, a zlib stream). **Writing images** (`user/img/pngsave.hpp`, header-only, integer only — freestanding apps use it): `pngsave::deflate` (LZ77 over 32 KB with hash chains, the fixed Huffman codes; zlib's wrapper or raw), `png_encode (px, w, h, alpha)` (RGBA / RGB, each row's best filter), `jpeg_encode (px, w, h, quality)` (baseline 4:2:0, the standard tables, an integer DCT), `gif_encode` (GIF89a: the exact colours up to 256, else a median cut; the clear pixels one transparent index; LZW), `bmp_encode` (24-bit), `on_white` (a pixel laid on white); `ZipOut` (add stored or deflated, finish) and `zip_find` (an entry's bytes and method). Paint's exports and its OpenRaster files use them.

TrueType text (`user/ft/`): the apps' FreeType — the upstream sources of `third_party/freetype-2.14.3` built lean by `user/Makefile` into `ft/libft.a` (`ft/onyx_ftoption.h`, `ft/onyx_ftmodule.h`: TrueType fonts only — `truetype + sfnt` —, anti-aliased — smooth —, hinted by the auto-hinter only — `autofit`, no bytecode interpreter —, their kerning read — GPOS too —; no compressed, web, bitmap, colour or variable fonts, no PostScript names; NetSurf keeps its own fuller build). FreeType wants a C library: an app using it is a **newlib** app (§5.1; Writer's rule in `user/Makefile` is the model). `#include "ft/fonts.h"` (header-only, one TU): `fnt::init ()` finds the families of `SD:/res/fonts` and `SD:/fonts` (each file's family name and style read from its own `name / head / OS/2` tables — no FreeType, three small reads), sorted; `fnt::count / name / find / styles`; `fnt::get (family, fnt::BOLD | fnt::ITALIC, size in 1/64 px)` a sized font (a style the family lacks is made: thickened, slanted) with its ascent, descent, height, underline place and thickness; `fnt::advance (f, cp)` — the design's advance, 1/64 px: exact at any size, so a line breaks at the same word at every zoom —, `fnt::kern`, `fnt::draw (cv, f, x64, baseline, cp, colour, clip)` — the glyph rendered once per quarter-pixel position, cached, blended with a slight gamma —, `draw_str / str_w` for Latin-1 labels. A character a font lacks comes from DejaVu Sans. `fnt::trim ()` drops the least used sizes when the cache is big — only at a moment no `Font *` is held (after a

redraw); `fnt::g_onTrim` lets the app forget its own. The screenshots' build compiles the same sources for the PC (`shots.sh`). **Vector shapes** (`wtk/vpaint.h`, in `libwtk.a`, integer only): a `VPath` gathers outlines — `poly`, `rect`, `rrect`, `circle`, `ellipse`, `hole` (a disc cut out), strokes `line`, `polyline`, `arc` (round ends and joins), `arrowHead` — in 1/16 px (`V(px)`), then `fill` (`cv`, `colour`, `alpha`) paints their union (the non-zero rule: every outline turned the same way, a hole the other way), anti-aliased (four sub-rows a pixel, the spans' ends to 1/16 px). Writer's toolbar icons are drawn with it (`user/Apps/writer/icons.h`). `wk_sin / wk_cos (degrees) × 16384`. **File-system providers (ABI v44)**: an app can serve a whole path prefix to every other app — `kapi_vfs_register ("XYZ:")`, then loop on `kapi_vfs_next (&req, 1)` and answer each request (`req.op = VFS_OP_OPEN / READ / CLOSE / LIST / SAVE / MKDIR / REMOVE / RENAME`, see `user/kapi.h`) with `kapi_vfs_reply (req.id, status, data, len)`; a SAVE's payload is read with `kapi_vfs_req_data`. Example: `user/bin/ftpfs.cpp` (FTP / FTPS). The ordinary file kapis then work on `XYZ:...` paths in every app, unchanged. `ftpfs.h: ftpfs_login / ftpfs_login_site` (hand a login to ftpfs, optionally remembered), `ftpfs_forget`, `ftpfs_load_sites` (the remembered servers of `SD:/etc/ftpfs.ini`). **Sound (ABI v46)**: `kapi_sound_acquire ()` first (1 = the output is yours; 0 = another app has it) — the output is released and silenced by `kapi_sound_release ()` or when your process ends. Then `kapi_sound_start (voice 0..15, milliHz, SOUND_SQUARE / SINE / TRIANGLE / SAW / NOISE, volume 0..255)` plays a note until `kapi_sound_stop (voice)` (-1 = all), and `kapi_sound_write (frames, n)` streams PCM (s16 L/R at `SOUND_RATE` 44100 Hz; non-blocking, returns the frames taken — loop with a short sleep while it returns 0) for audio / MIDI players. Example: `user/bin/tone.c`. **Low-latency sound (ABI v68)**: the output normally lags ~116 ms (1024-frame chunks, 4 rendered ahead). The owner may ask for less: `kapi_sound_config (chunk_frames, ahead)` (64..1024 frames, 1..4 chunks; 0 = the default) → the latency in frames, $(ahead + 1) \times chunk$ — $256 \times 2 = 768$ frames ≈ 17 ms, $128 \times 2 \approx 9$ ms (smaller means more DMA interrupts and no slack for a late core 1: try 256×2 first). Then keep little queued: with `kapi_sound_write`, write only while `kapi_sound_status`'s free frames show less than a chunk or two waiting. **The mapped ring**: `struct kapi_sound_ring *r = kapi_sound_map ();` (0 if you are not the owner) — 8192 frames in shared memory that the kernel mixes (with the voices and the stream) until you release the output. `kapi_sound_ring_write (r, frames, n)` → frames taken (`kapi_sound_ring_free (r)`: the room). It makes no kapi call, so **code on an app core** (`kapi_core_run`) can fill it: the audio needs no pump thread on core 0. `r->dry` counts the underruns; `r->rd` moves as the kernel plays — a thread can sleep on it with `kapi_wait_word (&r->rd, old, ms)` (§5.2). All of this is undone by `kapi_sound_release` or the process's end. Example: `user/bin/ringtest.c` (a tone from an app core at 256×2). **FM instruments (ABI v47)**: fill a `struct kapi_fm_instrument` (2 operators, OPL2-style parameters, see `kern/kapi_abi.h`), `kapi_sound_instrument (voice, &ins)`, then `kapi_sound_start (voice, milliHz, SOUND_FM, volume) / kapi_sound_stop (voice)`. The FM Song formats (`.FMS / .FMI`) and their conversion are in `user/Apps/fmtracker/fms.h` (portable; host tests: `sh tools/tests/run_fms_test.sh`). **Held keys (ABI v48)**: key events only report presses; an action game polls `kapi_key_held (KEY_LEFT)` (also `KEY_RIGHT/UP/DOWN, KEY_ENTER`, 27, ' ', 'a'..'z' = the US position of the key, '0'..'9') every frame — 1 while held and your window has the keyboard (USB keyboard and VNC). Returns 0 on an older kernel. **USB gamepads (ABI v50)**: `#include "gamepad.h"` — `pad_buttons (-1)` = the `PAD_*` buttons held on every pad (or 0..3 for one): `PAD_UP/DOWN/LEFT/RIGHT, PAD_A` (bottom face button), `PAD_B` (right), `PAD_X` (left), `PAD_Y` (top), `PAD_L/R/L2/R2, PAD_SELECT/START, PAD_L3/R3, PAD_HOME`; `pad_read (i, &in)` adds the sticks (`in.lx/ly/rx/ry`, -1000..1000), the USB ids and the raw bits. Nothing is pressed while your window has not the keyboard. The pad's raw state (`kapi_pad_state`) goes through

SD:/etc/gamepad.ini (a [vvvv:pppp] section per pad model, [default] for the other generic pads; analog triggers as `l2_axis / r2_axis`; written by the Gamepad app, `padconf`) or the built-in mapping (pads Circle knows). Used by gbemu, gamelib, BASIC (PAD, STICK, STRIG). Host test: `sh tools/tests/run_gamepad_test.sh`. **USB MIDI input (ABI v68)**: class-compliant USB MIDI devices (keyboards, interfaces) are found when plugged in, at boot or later. `kapi_midi_read (ev, max)` takes up to `max` queued `struct kapi_midi_event` (oldest first; never waits) → how many: `time_us` (arrival, the kernel's μ s clock — `kapi_clock_us ()` reads the same clock, also on an app core), `cable`, `status`, `data1`, `data2` (length of them valid: 1..3; a SysEx arrives in 3-byte pieces), `device` (its `umidiN`). One queue for the whole system (256 events; the newest are dropped when nobody reads): one reader at a time. `kapi_midi_devices ()` → attached now. Poll it from your pump or a thread (every 1–2 ms for live playing). Example: `user/bin/miditest.c`. **The GPU (ABI v52)**: `kapi_gpu_info (buf, cap)` (1 = the V3D is usable; the first call brings it up) and `kapi_gpu_draw (verts, n, clear, pixels, w, h, stride)`: a triangle list of `struct kapi_gpu_vertex { float x, y, z; unsigned char r, g, b, a; }` in normalized device coordinates (y up, z – 1 near ... 1 far), depth-tested and Gouraud-shaded by the GPU into the app's pixels (e.g. its window canvas: `canvas.px`, `canvas.stride`). The app transforms and lights its geometry itself (floats: build it with `CXXFLAGS_FP`, like `teapot`); keep a software path for `r < 0`. See `user/Apps/teapot` (`teapot.h` is portable): `tools/tests/teapot/teapot_host.cpp` renders it on a PC) and docs/02 §15. **The GPU's full pipeline (ABI v53)**: textures — `int t = kapi_gpu_texture (-1, argb, w, h, stride);` (0xAARRGGBB, up to 2048 × 2048, 256 handles; the same call with `t` replaces the pixels, with `pixels = 0` frees it; freed anyway when the app ends) — and `kapi_gpu_render (&frame, verts, nv, batches, nb)`: `struct kapi_gpu_vertex3 { x, y, z, w, s, t, r, g, b, a }`, each `struct kapi_gpu_batch` draws `count` vertices from `first` with its own **4 × 4 matrix** (row by row, clip = $M \cdot (x\ y\ z\ w)$: build model × view × projection once per object on the CPU, the GPU transforms every vertex; `KAPI_GPU_B_NOMATRIX` if they are already in clip space), texture (–1 none), `KAPI_GPU_B_*` flags (depth test / writes, culling, blending alpha / add / multiply, linear filter, wrap). Draw opaque batches first, then the blended ones with `KAPI_GPU_B_NOZWRITE`. `frame.flags = KAPI_GPU_F_KEEP` draws over the pixels. **v54**: each vertex also carries `r2 g2 b2 a2`, a colour **added** after `texel × colour` (0 = unchanged; the N64 emulator's colour combiner is `texel × c1 + c2`), and `KAPI_GPU_B_ALPHATEST(t)` discards the fragments whose alpha is below `t / 255` (cut-out foliage, fences, text) without blending or depth sorting. **ABI v61 — your own shaders**: `kapi_gpu_program (-1, &prog)` uploads a vertex, a coordinate and a fragment shader (V3D 4.2 QPU words, generated at run time with `user/v3d/qpu.h`: `qpu::Prog p; p << qpu::I ().a (V3D_QPU_A_FADD, rf (3), r1, r5).ldvary (r0); ...`; every app links `v3d/libv3d.a`; test them on the PC with `tools/qpu/qpusim` — `tools/tests/run_qpu_test.sh`), then `kapi_gpu_render2 (&frame, verts, nv, stride, batches, nb, uniforms, nuni)` draws `struct kapi_gpu_batch2` batches of vertices of `stride` floats (the clip-space `x y z w` first, clipped by the kernel) with their program, uniform ranges, up to 8 textures (the kernel writes each one's TMU words at `fsUni + texUni[i]`), blending (`KAPI_GPU_BLEND2`), write mask and scissor. The vertex shader writes `Xs Ys` (24.8 fixed point: $x \times w/2 \times 256 \dots$) `Zs` $1/Wc$ then the varyings; the coordinate shader `Xc Yc Zc Wc Xs Ys`; the fragment shader ends with its TLB writes after the last thread switch — see the v53 shaders in `kernel/sys/v3d_shaders.qasm` for the recipes, or take the ready-made ones of `user/v3d/shaders.h` (`passVS / passCS`: the position as it is, the other floats handed on as varyings; `flatFS`, `varyFS`, `texFS`; `viewUniforms (w, h)`). `/bin/v3dprog` checks them on the Pi (PASS / FAIL); `tools/tests/run_qpu_test.sh` checks the same programs on the PC (the instruction restrictions, then the fragment shaders in the simulator). The GameCube's TEV is

generated by `user/v3d/gxtev.h` (`gxtev::build (config, shader)`: the stages in integers as the hardware, the texture lookups, the alpha test, the EFB format -> QPU code + the uniforms and varyings the draw must give); `tools/tests/v3d/gxtev_test.cpp` checks thousands of random configurations in the simulator against `user/v3d/gxtev_ref.h` (the GLSL TEV of `gxgl.cpp` in C++), `/bin/v3dprog` a few on the GPU. `gcemu` draws with them: `user/Apps/gcemu/gxv3d.h` is its `gc::GxGpu` -- the app core records each draw (the vertex stage of `gxgl.cpp` in C++, the TEV program by its configuration, the uniforms, the state), the main thread gives the frame to `kapi_gpu_render2`; `tools/tests/gc/gcv3d.cpp` runs a `.dol / .iso` on the PC with the same recorder and draws its frame with a software V3D (the shaders in `qpusim`) into a `.ppm` (`GCV3D_EVERY=n` more frames, `GCV3D_DUMP=1 / 2` the batches / their programs, `GCV3D_SAVE=<f.gxf>` the last frame dumped, `GCV3D_SKIPLOG=<file>` each draw the recorder leaves out and why -- `Rec::skipWhy`, `Rec::onSkip`; `GC_CARD=<f.sav>` a memory card, read only, `GC_PAD="f0-f1:hex;..."` the pad's buttons held from field `f0` to `f1`, as `gcrun`'s: the games' menus scripted -- The Wind Waker's first steps on Outset: Start at 900, A at 1050, 1200, 2200, 2300, 2400, Start at 2500, A at 2600, then A every 60 fields to 22000; `GC_HASH=n`, in `gcrun` too: MEM1's hash every `n` fields, two runs compared -- the GL and the TEV renderers gave the same game, so a difference was the drawing's; `GCV3D_VHASH=1`: each finished frame's recording hashed, a line a frame -- a change of the recorder or the vertex decoder checked bit for bit over a whole run). The recorder's vertex loop and `GxPrimitive`'s decoder work from plans made once a draw (the matrices of the last index, the lit channels' lights, the texgens, the varyings; the attributes present with their formats and arrays, a vertex template) with the same arithmetic as before. The recorder's plan (`Rec::Plan`) is **kept from a draw to the next**: `planState` (the TEV's configuration and program, the textures, the batch and its uniforms, the vertex stage's plan) is made again only when `GxState::serial` changed, `planXf` (the lights, the texgens' post-transform matrices) when `Machine::xfSerial` did (every XF memory write, the indexed loads too), and the matrices of the vertices' indices stay while the XF memory is the same; a draw of the same plan right after its last one goes on in that batch. The Wind Waker's draws are 4.6 vertices on average, ~90 % of them with the last one's state and XF memory: its recorder 15.7 -> 8.2 ms a field on the Pi (the GX core 92 -> 61 % busy). A colour's varying (`/ 255`) comes from a table (the colours are integers 0..255) and a texture coordinate's `/ size` is a product when the size is a power of two (both exact: the recordings bit for bit the same, `GCV3D_VHASH` over 2400 fields). The GX's reset keeps its state's serial and `xfSerial` going (a plan made before must not match). A triangle whose three vertices are before the near plane or behind the eye (the kernel's first two clipping planes, `V3DClipDistN`: the kernel would drop it) is not recorded -- ~10 % of The Wind Waker's, copied for nothing by the recorder, `Out::prepare` and the kernel before (the pictures the same: `gcv3d`; F12's fourth line counts them). The recorder keeps the vertices in the clip space of the whole EFB (640 x 528) and the scissors in its pixels; the frame's XFB copy rectangle, known at its end (`Frame::rect`), is applied when it is drawn (`frameView`, `frameScissor`: `Out::prepare`, `gcv3d`) -- the copy registers of the draw's time were used before, and a half-size copy to a texture in the middle of a frame (The Wind Waker's effects) framed what followed as a quarter of the picture (the camera "in Link's head"). **On the Pi**, `gcemu`'s F12 third line says what the TEV renderer did (`gpu_render2`'s result, the batches drawn / recorded, the ones left out: program refused, texture missing, nothing visible, over the vertex limit), F9 dumps the frame shown into `SD:/gcdump/frame_<n>.gxf` (the recorded frame, its programs and textures, and the picture the GPU made), `--diag[=<folder>]` writes F12's lines of every second into `<folder>/diag.txt` (saved every 5 s), dumps a frame every 60 s (the machine waits meanwhile: a few seconds over FTP) and quits after 200 s; `--statlog[=<folder>]` writes the same lines into `<folder>/statlog.txt` without the dumps and

the end (the last ~200 KB kept, saved every 5 s: a measure of a game played, F12's lines whole whatever the window's width); `--nodraw` (a measure) records the TEV frames but neither prepares nor draws them (the window stays still): the machine's speed without the display's work beside it; F12's fourth line is where a field's time went on the app core — the GX (`Machine::timeFifo`, all in), its primitives (`timePrim`: decoding, state, textures, the recorder), the textures (`timeTex`), the recorder (`Rec::drawTicks`) — and a frame's on the main thread (`Out::prepare`, `gpu_render2` — the kernel splits the latter in `kmsg` every 256 frames: `render2`, a frame: `clip ... lists ... GPU ... target ...`); the counters read the ARM's clock (`gcClock`, nothing on other hosts) — the folder may be an FTP one (`--diag=FTP:<pc>:<port>/<dir>`) so the files land on the PC at once (`--tevbuf`: the frames drawn into a buffer of `gcemu`'s, then copied — the GPU no longer writing the window's pixels itself). F10: the game's frames a second (its XFB copies) and the speed alone, in a corner. `--pmu[=e1,e2,e3,e4]`: the app cores' performance counters, a fifth line (millions of cycles a field, instructions a cycle, and a thousand instructions' L1D refills, L2 refills, branch mispredictions -- the events, hex, default 08, 03, 17, 10), the CPU's part and the GX's, then the share of the machine's cycles inside the JIT's code (`in JIT`, its helpers too) and why its runs ended a field (`ends`: a VI line, DI, the audio DMA, AID, DSP, EXI, card, an AI sample; then the JIT's entries from C); `--noren` (a measure) prepares the TEV frames but does not draw them. `--jitprof` (with `--diag`): the JIT's profile every 60 s -- `jitprof.txt` (the costliest blocks, the host instructions each guest instruction costs, the instructions left to the interpreter, the slow memory accesses) and `jitprof.bin` (the blocks' code: `python tools/gc/jitprof.py jitprof.bin --top N --ppc <objdump> --a64 <objdump> --host` disassembles both sides). The machine's cores set FPCR to 0 (IEEE: to nearest, no flush-to-zero; nothing sets it at boot): F12 shows `FPCR was ...` when it was not. **The GX on its own core** (when a second app core is free -- `netcore=0` in `cmdline.txt` leaves cores 2 and 3 to the apps): the machine's core writes the FIFO (the write-gather pipe's bursts: `gatherFlush` publishes the PI's write pointer), the GX's core runs it meanwhile (`Machine::gxStep`: the commands up to that pointer, then `gxDoneW`), reading them in place (a command cut by the FIFO's end only is copied). The machine waits for it (`gxSync`) where the game can see its progress: a CP / PE / PI-FIFO register, a polling loop the JIT skips (the game waiting for the GPU: `idleHit`), a full FIFO (`gxRoom`); the PE's token / finish come back through `gxIrqBits` (raised on the machine's core); `gxLock` keeps the frame's flip and the texture pool from the main thread's `Out::prepare`. F12's fourth line gives the GX core's busy share and the machine's waits; `--gxone` keeps the GX on the machine's core, to compare. `gcv3d --replay <frame.gxf> <out.ppm>` draws a dump on the PC at the Pi's size next to the Pi's own picture (`out_pi.ppm`): the same picture → the GPU is right, the recording is the question; another one → the GPU side. `v3dprog ww <w> <h> <flags> <percent> <variant>` draws The Wind Waker's first frame (its TEV program, one quad) into a `w × h` target — the experiment that found the `P_FS_FINAL` hangs (docs/02 §15; variants: +1 the plain varyings shader, +2 RGB8, +4 no scissor, +8 / +16 the flat shader final / with a thread switch). Example: `user/Apps/gpudemo`. The kernel's shaders are QPU assembly (`kernel/sys/v3d_shaders.qasm`): after editing, `cd tools/qpu && make` re-assembles and checks them into `v3d_shaders.inc` (committed; the kernel build does not need the tool). docs/02 §15. **App cores (ABI v51)**: an app may take a whole core (2 or 3) for a function of its own — `int c = kapi_core_acquire ();` (−1: none free), `kapi_core_run (c, fn, arg, stack_top)` (the stack is the app's memory, 16-byte aligned), poll `kapi_core_state (c)` (`KAPI_CORE_IDLE` once `fn` returned, `_RUNNING`, `_FAULT`), `kapi_core_release (c)` (stops `fn` if needed; done at exit anyway). **fn makes no kapi call and no allocation** (neither the kernel nor `malloc` is multi-core safe) and never masks the interrupts: it computes and talks to the main thread through memory — `volatile` flags with barriers (`dmb ish`), `wfe` to wait, the main

thread's `sev` to wake it, the clock read directly (`cntpct_e10`). Stop it cleanly with a flag it polls. Keep working without a free core (run the same function on the main thread). **Emulators:** `#include "emucore.h"` does all that — `ec_init (&ec, w, h, frame_fn)` (acquires a core if one is free), `frame_fn (EmuCore *)` runs one frame, writes the picture into `ec_back ()` then `ec_publish ()`, pushes its sound with `ec_audio_push`; the main thread `ec_request (&ec, n)`, `ec_pump` (frames made here when there is no core), `ec_take / ec_front` (the latest picture: a triple buffer, nobody waits), `ec_audio_pop`, `ec_hold / ec_resume` around anything that touches the machine (reset, battery save), `ec_shutdown`. Two rules the app core imposes: **its code allocates nothing** (the heap, `user/umm.h`, is not shared safely between two cores, and `kapi_sbrk` from an app core grows the heap of whatever task core 0 is running — make every buffer before, as `gc::Machine`'s texture pool of `TEX_POOL` texels); and **take the new picture before asking for the next frame** (in the main loop: `if (ec_pending (&ec) == 0 && ec_take (&ec)) ... first, then ec_request`) — the other way round, a machine slower than real time always has its next frame asked first and the window is never drawn again. What reads the machine (its frame, its textures) runs while it waits, or under a lock it respects; what only reads a copy may run while it works: `gcemu` asks for the next field, then makes the kernel's batches of the TEV frame (`Out::prepare`, under the GX's lock: the recorder's finished frame and the textures kept still; it marks that frame `held`: of the recorder's three frames, the GX builds into the other two meanwhile) and draws it (`Out::submit` → `gpu_render3`, `kapi v62`: the kernel reads the frame's vertices where they are and frames them onto the XFB copy's rectangle on the way — `kapi v63`: the recorder's three frames are `gpu_vbuf` memory, drawn in place, nothing copied, the frame framed there once (`Frame::framed`: drawn again with `view 0`; F9's dump then says the whole EFB); an older kernel: the vertices copied into one array for `gpu_render2`, as before) while the field runs — two fields asked for at once when behind real time (the drawing, ~30 ms with ~100k vertices, is longer than a field: the machine, its field done, no longer idles until the next request; the picture is then taken after the second) — and not again when the same frame is already in the window (a 30 fps game: every other field). While a field runs the main thread yields (its end seen at once), but naps (`kapi_msleep (1)`: to the scheduler's next tick) while that end, expected from the last second's time a field, is further than 12 ms — a yield in a loop kept core 0 busy (heat: the Pi 4 throttles its clock above ~80 °C, `kmsg's power`: lines). See `gbemu / gbaemu / nesemu / snesemu`; `/bin/coretest` exercises the raw `kapi`. **Game kit** (`user/game.h`): `GameView` (a full-window widget: `paint`, `press / release / move edges`, `key`, `tick (dt)` at ~60 Hz), `GameRoot` (ticks it, routes every key to it), sound effects on voices 12..15 (`sfx (hz, ms, wave, vol)`), `sfx_later` for jingles, `sfx_win / sfx_lose`, `sfx_set_mute`; the output is acquired on first use), `rng / rng_n`, text helpers (`gtext`, `gtext_c` centred with a shadow, `gtoa`, `gcat`). Cards (`user/cards.h`): `card_face / card_back / card_slot` (64×88) and the bouncing-cards victory animation (`win_start / win_step`). Used by invaders, pipes, solitaire, freecell (Arkanoid is now the BASIC game). **Host test:** `sh tools/tests/run_games_test.sh [INVADERS ...]` builds each game on the PC against a fake `kapi` table (`tools/tests/wtkhost/host_kapi.h`: every slot a stub, files from `sdcard/`), plays a scripted scenario (UBSan) and saves real screenshots to `/tmp/onyx_games`. **Rich Text Format** (`user/rtf.h`): `rtf::load (box, data, len)` parses an RTF document into a `RichTextBox` (styles, colour table → the 16-colour palette, `\'hh / \uN` → Latin-1, skipped destinations), `rtf::save (box, out, cap)` writes it back; `rtf::is_rtf`. Used by `rtfview` (Writer reads and writes RTF itself, with everything: `Apps/writer/fileio.h`). **Writer** (`user/Apps/writer/`, one TU: `main.cpp` includes the rest) — `doc.h` the document (paragraphs of code points, each with an index into the table of character formats — font of the font table, size in half-points, flags, colour, highlight, a field (`FIELD_CHAR U+FFF9` whose format names a `Field`: page, pages, date / time with its picture, a merge field) or an image (the character

U+FFFC whose format names the image and its size) —, each paragraph its `ParaFmt` — style, alignment, indents and spacing in twips, line spacing, list, page break, keep with next / lines together / widow control, tab stops (position, alignment, leader) and, in a **table**, the table's index and the cell's row and column; the **stories** — the body, the header, the footer, the first page's own — one of which is edited at a time (`d.p`, `doc_story` swaps them); a **table** is a run of paragraphs saying their cell, the `Table` itself (columns' widths, rows' heights, the cells' spans, shading, lines, the heading row) a value in `d.tbl` replaced whole by an edit; the tables only grow, so an edit's undo just puts the paragraphs it copied back: `doc_begin` / `doc_end_edit`, typing coalesced), `layout.h` (lines at the zoom in 1/64 px from the fonts' design advances in boxes — the text's width or a cell's —, each character's x kept — `Para:xs` — for drawing, hit-testing and the caret alike; tab stops, fields' texts, lists' numbers; the headers and footers placed first — the body's top and foot follow them —, then the **pages**: a `Pager` keeps a heading with its next line, widows and orphans off, a table broken between its rows — the rows its cells span together — with its heading row repeated), `edit.h` (the selection, the edits — the tables' rows, columns, merges and splits as one undoable edit of the table's paragraphs and its `Table` —, the stories, the fields, the page numbers, the **table of contents** (laid out twice: its own place moves the pages), the clipboard — the system's text plus the piece of document kept here —, find), `view.h` (the page view widget: drawing — the tables' shading and lines, the fields shaded, the leaders, each page's header and footer, the body greyed while one is edited —, caret blink without a redraw, mouse, keys, images' resizing, a table's column border dragged), `ui.h` (tool buttons, pick boxes dropping a `ListPopup`, the size box, colour popups, the ruler — a table's columns —, the status bar), `icons.h` (VPath icons), `fileio.h` (RTF in / out — tables `\trowd...` — their lines as the cells give them: every edge, the rows' only, none —, headers and footers, fields `{\field}`, tab stops, the TOC's field, pictures as `\pict\pngblip` / `\jpegblip`, a PNG made when the image came as something else —, text, HTML), `xml.h` (a pull reader over a zip entry — wtk's `img_inflate` —, the units; the zip written with the PNG writer's deflate), `docx.h` (WordprocessingML: styles with their inheritance and the theme's fonts, numbering, tables — grid, spans, vertical merges —, simple and complex fields, the headers and footers, the section, images, `w:docVars` for the mail merge's data), `odt.h` (ODF: named and automatic styles, lists, tables — spanned and covered cells —, frames, fields with their data styles, the TOC, the master page), `merge.h` (the **mail merge**: the data is a Cardfile form read with Cardfile's own `model.h`; the fields filled per record into a copy of the letter read back from its bytes; `writer --merge JOB` — the job file's keys at the top of `merge.h` — makes a merge's documents for Cardfile and Ledger: a job's `lines` form — a record a line of a document — has the letter's table row holding `Line...` fields repeated for each of its records, `merge_lines`; one file written is shown without a message), `dialogs.h`. **Host test:** `sh tools/tests/run_writer_test.sh` (`tools/tests/writer/files_test.cpp`: a document with all of it written as RTF, `.docx` and `.odt` and read back the same — each through the others too —, and with LibreOffice installed — `soffice` — our `.docx` and `.odt` converted by it and read back; UBSan, `VG=1 valgrind`); `tools/tests/writer/conv.cpp` converts a file by the names' extensions (`conv a.docx b.odt`). The samples (`SD:/docs/writer-tour.rtf`, `new-year-letter.rtf`) are made by `tools/gen_writer_sample.py`. **Paint** (`user/Apps/paint/`, a freestanding integer app) — `pdoc.h` (up to 32 layers of 0xAARRGGBB pixels, straight alpha, bottom first; the composite kept and recomputed by rectangles, a floating selection and a shape's preview composed with the current layer; undo: a stroke keeps the 64 × 64 tiles it touches as they were, a change of size or of layers the whole picture), `raster.h` (stamps, Bresenham lines, Zingl's ellipse in a box, polygons filled by pixel centres, the shapes inscribed in their box's ellipse, flood fill, flips, quarter turns, scaling), `pview.h` (the canvas widget: zoom, grid, checkerboard, the tools, the

floating selection), `pui.h` (the ribbon — cells laid out in code, their tips set as the pointer moves —, the layers' panel, the status bar, the `VPath` icons), `pfile.h` (OpenRaster in / out, the pictures, the exports). A host check: random strokes, shapes, fills, layer changes, turns and moved selections undone then redone must give back the same pixels. **Cardfile** (`user/Apps/cardfile/`, one TU: `main.cpp` includes the rest; integer only, no libc) — `model.h` the document: a form (`Field`: display name, column name, type `FT_*`, decimals, choices) and its records (an array of heap strings a record, the empty value shared), each value kept in one canonical text a type (integer `-12`, decimal `12.50` at the field's decimals — a `long long` scaled by 10^{decimals} when computed —, date ISO `2026-09-29`, colour `#3366CC`, yes / no `yes` or empty); the values read as typed (`parse_num`: `.` or `,` as the point, group separators; `parse_date`: `D/M/Y`, `D.M.YY`, `D/M`, `DDMMYYYY`, ISO), shown (`value_show`), made (`value_new`), compared (`value_cmp`: numbers, dates, a choice by its list's order, text folded — Latin-1 case and accents — with its runs of digits compared as numbers); the file (`doc_write` / `doc_read`: an INI-like head, the records tab-separated with `\t \n \\` escapes, read leniently — the format is described at the top of `main.cpp` and in docs/04); a type changed (`convert_count` / `convert_field`: through the value's shown text); the fields inserted, removed, moved (`doc_remap`: every record laid out again); CSV (`csv_write`, `csv_read` with each column's type guessed); the order (`build_order`: the search's words, a stable merge sort, empty values last). `widgets.h`: the editors — `LineEdit` (any length, selection, clipboard, a filter of the characters; over a `TextCore` it shares with `MemoEdit`, wrapped at the words), `DateEdit` + `CalPopup` (wtk's Calendar), `ColorEdit` + `ColorPopup`, `ChoiceBox` + `PickList` (a scrolling list over the window), `YesNoBox` —, the toolbar, the view switch, the search box, the navigator (`NavBar`), `VPath` icons. `formview.h` (the card: an editor a field, the commit and its validation, Tab order, scrolling), `listview.h` (a `DataGrid`), `designview.h`, `app.h` (the state the views share). Undo keeps the whole document written before each change (100 steps, 24 MB at most; one control's edits coalesced into one step). **Record ► Mail Merge** (`MergeBox`, `cmd_mail_merge`): the records to merge written as a `.card` (`SD:/apps/cardfile.app/merge.card`), a job file beside it, then `kapi_exec` of `writer --merge JOB` — `Writer's merge.h` reads both; the form's `merge` key keeps the letter. **Host test**: `sh tools/tests/run_cardfile_test.sh` (the values as typed, a round trip byte for byte, a file edited by hand, a type changed, fields moved, CSV, the order; ASan + UBSan). **Ledger** (`user/Apps/ledger/`, one TU: `main.cpp` includes the rest; integer only — money in **cents** (`money`, a `long long`), VAT rates in hundredths of a percent, quantities in thousandths, dates `yyyymmdd` —; Cardfile's `model.h` and `widgets.h` are reused: strings, `Out`, the editors). **The engine** (plain C++, the same on the PC, no wtk): `core.h` (money and dates typed and shown the Belgian way, the checks — VAT numbers, IBAN, BIC, structured communications `+++...+++` —), `model.h` (the *Book*: company, fiscal years, accounts, parties, journals, entries of lines — account, amount, party, VAT code, role, due date, matching group —, VAT returns, the commercial documents; matching: a group of lines adding up to zero), `pcmn.h` (the PCMN in French and Dutch, generated by `tools/ledger/gen_pcmn.py` from `pcmn.tsv`), `setup.h` (a new company's chart and journals, the fiscal years added, closed — the result appropriated —), `vat.h` (the VAT codes — Odoo's Belgian ones: a code, its rate, its side, the grids its base and tax go to, a reverse charge's both sides —, the grids of a period, Intervat's checks, the return's XML — `NewTVA-in v0.9` —, the settlement, the customer and intra-Community listings' XML), `post.h` (an *Invoice* and a *Statement* as typed ↔ their *Entry*, the checks, the numbers, the structured communication made from a sale's number, the matches applied), `reports.h` (a *Report*: rows of cells — journals, general ledger, trial balance, balance sheet and income statement by the PCMN's headings, balances, ages, a party's account —), `fileio.h` (the `.ledger` file: its format described at its top; read leniently, written whole), `commerce.h` (quotes,

orders, delivery notes, purchase orders: numbered by kind and year, their totals, one made from another, the invoice made from one), `coda.h` (Febelfin's **CODA** read — the records 0, 1, 21 / 22 / 23, 8, 9; a globalisation's details skipped —, a statement made of it: the parties and the invoices paid found by structured communication, IBAN, name; a writer for the demo and the tests), `sepa.h` (the suppliers' open lines, **pain.001.001.09** written: hybrid addresses, the EPC's Latin set, a structured communication as `SCOR / BBA`), `print.h` (the merge's data for a document's template: the fields, the lines, the VAT's detail with the legal mentions in French, Dutch or English — a party's language —, then `writer --merge`), `export.h` (a Report as RTF — A4, landscape when wide, a header and page numbers, tables — or `.xlsx` — `img/pngsave.hpp`'s `ZipOut` — or CSV, opened in Writer or the Spreadsheet). **The window:** `ui.h` (the Page a view is — its header, `sync` when the books or the year shown change —, the icons as `VPaths`, `FlatButton`, `HeadRow`, `Segmented`, pills, tiles, `TickGrid`, `AskBox`, `FormBox`), `pick.h` (`PickEdit`: a party, an account, a VAT code found as typed, its suggestions below), `editgrid.h` (`EditGrid`: a document's lines typed in place over an `EGModel` — Tab / Enter / arrows, F4, a line added at the end —), `docs.h` (`DocPage`; the invoice, the statement — its open items ticked, a CODA statement to complete —, the misc. operation), `lists.h` (the overview, the journals' lists, the parties' cards and accounts, the chart), `pages2.h` (reports, VAT, settings — the templates' pane —, the new company), `commerce_ui.h` (the quotes' list and page, *Next step*), `payui.h` (the SEPA dialog), `main.cpp` (the side bar, the pages made when first shown, the books' file saved at each change, the CODA import's queue, the menus). **Host test:** `sh tools/tests/run_ledger_test.sh` (`tools/tests/ledger/engine_test.cpp`: money, dates, the checks, invoices and credit notes, a statement, the VAT grids, matching, the file byte for byte, the reports, the year's end, the commercial documents, CODA read and made a statement, a SEPA file — 200 checks, ASan + UBSan —; the XML files validated by `xmllint` against the official schemas in `tools/tests/ledger/xsd/`: Intervat v0.9, ISO 20022 `pain.001.001.09`). **Generators:** `tools/ledger/make_demo.cpp` (the demo company through the engine itself — `sdcard/docs/demo-company.ledger` — and its next CODA statement, `demo-bank-statement.cod`: see its header), `tools/ledger/gen_templates.py` (the documents' templates in French, Dutch, English, and `templates/fields.card`), `tools/ledger/gen_pcmn.py`. **Spreadsheet** (`user/Apps/sheet/`, a **newlib** wtk app — `FreeType` and `libm` — built as Writer is; one TU: `main.cpp` includes the rest, a chain of headers each including the one before, all in `namespace ss`; `app.txt` asks for a **4 MB stack**: the formulas are evaluated recursively). **The engine** is plain C++ over `libc`, the same code on the PC: `core.h` (a growing `Buf`, UTF-8 — the cells' text is UTF-8, wtk's font and the keyboard Latin-1 with the euro at 0x80: `latin1_cp`, `latin1_to_u8`, `u8_to_latin1` —, numbers ↔ text, Excel's serial dates, the *Arena* a formula's temporary values live in, released after each cell), `book.h` (the *Book*: its sheets, the interned *Styles* — a cell keeps an index —, the defined names; a *Sheet*: the used cells in a hash `CellMap` by row and column, the widths / heights / hidden flags, merges, frozen panes, charts anchored to a cell, conditional formats, the *AutoFilter*; a *Cell*: what was typed — number, text, formula — and its value), `formula.h` (Excel's syntax read into tokens — kept to print the formula back as typed — and a tree over them; each reference its row / column and \$ flags; `formula_copy` (`dr`, `dc`) moves the relative parts, the shifts of inserted / deleted rows and columns move every reference, a deleted one becomes `#REF!`; the names' table), `eval.h` (the tree walked: *IF* and its kind lazy, operators over numbers / texts / arrays element by element, a function taking one value lifted over an array; `recalc` recomputes every formula at each change, a formula computing the cells it reads first — the book's `epoch` marks what is done —, a chain deeper than `MAX_DEPTH` pushed on an explicit stack, a cell met again on it `#CIRC!`), `numfmt.h` (Excel's format codes compiled once — `fmt_get` caches them — and applied), `input.h` (what a typed entry is: number,

percentage, amount, date, time, boolean, error, text — with the format it implies), `fn_core.h` / `fn_more.h` (the 237 functions, Excel's names, arguments and results), `funcs.h` (the table FNS: name, arguments, flags — lifted, lazy, volatile —, category, the argument list and the line the tips and Insert Function show), `ops.h` (what the window does to the book: a cell set, a value shown, styles over a range, rows / columns inserted, deleted, sized, cells moved / copied / filled / sorted / merged, sheets; the AutoFilter), `undo.h` (before a change, the range, sheet or book it touches written in a small binary form; Undo writes the present the same way for Redo; 100 steps, 64 MB). **The files:** `xml.h` (a pull reader, the escaping), `xlsx.h` (Excel's `.xlsx`, read and written — the archive through `img/pngsave.hpp`'s `zip_find` / `ZipOut` and `img_inflate`; under `SHEET_APP` the files go through `kapi`, on the PC through `stdio`), `ods.h` (LibreOffice's `.ods`, read: OpenFormula turned into Excel's syntax, the number styles into format codes), CSV in `main.cpp`. **The window:** `condfmt.h` (the rules' look for a cell — their figures over the range computed once a recalculation), `ui_base.h` (Writer's toolbar look, the icons Writer lacks), `render.h` (a cell drawn: FreeType fonts — the Office families mapped to the card's —, the formatted value, wrap and overflow, fill, borders, merges), `chart.h`, `grid.h` (GridView: headers, up to four panes, the selection, the fill handle, the charts, the in-place editor with its coloured references, pointing, the functions offered and their tips, the AutoFilter's buttons), `bars.h` (NameBox, FormulaBar, SheetTabs, StatusBar), `dialogs.h`, `main.cpp` (the menus, the commands, the clipboard — the cells kept here, their text on the system clipboard: a paste from elsewhere is read as a tab-separated table —, the files, `docguard.h`, the recovered workbook). **Host test:** `sh tools/tests/run_sheet_test.sh` — `tools/tests/sheet/engine_test.cpp` (formulas, functions, formats, typed entries, references moved, names, undo) and `files_test.cpp` (`.xlsx` round trips; with LibreOffice installed, its `.ods` and `.xlsx` of the same workbook read back alike); ASan + UBSan. The sample `sdcard/docs/cafe-2026.xlsx` is made by `tools/tests/sheet/make_sample.cpp` (built with the engine itself: see its header). **Game Boy / Color core** (user/`gb/gb.h`, `gb/libgb.a`, linked into every app): `gb::Machine` — `load` (`rom`, `size`) (CGB mode from the header), `runFrame` () → `fb` (160×144, 0x00RRGGBB), `setButtons` (`gb::BTN_* mask`), `setAudioRate` (`hz`) + `audioRead` (`lr`, `n`) (s16 stereo), `setSaveRam` / `sram` / `sramDirty` (battery saves), `setDmgPalette`. The SM83 CPU with instruction timing, a scanline PPU (DMG + CGB), timer, OAM DMA / HDMA, MBC1/2/3(+RTC)/5, the 4-channel APU; integer only, no libc. Used by `gbemu` (the emulator: paced by the audio queue, else the clock; the machine on an app core through `emucore.h`) and `gamelib` (the library: title-screen thumbnails made headless). The ROM windows are pointers set when the game switches banks (`mapRom`), the background and window are drawn a tile at a time. **Host test:** `GB_TEST_ROMS=<unzipped c-sp game-boy-test-roms>` `sh tools/tests/run_gb_test.sh` (Blargg `cpu_instrs` / `instr_timing` / `halt_bug`, `dmg-acid2`, `cgb-acid2` pixel-exact); `tools/tests/gb/gbtest.cpp` `<rom>` `<seconds>` `[out.ppm]` `["t:mask,..."]` runs any ROM headless. **Game Boy Advance core** (user/`gba/gba.h`, `gba/libgba.a`): `gba::Machine` — the same shape as `gb::Machine` (`load`, `runFrame` → `fb` 240×160, `setButtons` (`gba::BTN_*`), `setAudioRate`, `audioRead`, `setSaveRam` / `save` / `saveSize` / `saveDirty` / `saveType`). The ARM7TDMI (ARM + Thumb, the two prefetched opcodes kept: self-modifying code sees them), the memory map with WAITCNT wait states and open bus, the frame loop driven by events (line phases, timer overflows), DMA (immediate, V/H-blank, sound FIFO), 4 timers (cascade), the PPU a line at a time (modes 0-5, affine, sprites, windows, blending, mosaic), the PSG + Direct Sound A / B, the BIOS calls done in C++ (`gba_bios.cpp`: no BIOS image), SRAM / Flash 64-128 KB / EEPROM (found from the ROM's library string). Used by `gbaemu` and `gamelib`. **Host test:** `GBA_TEST_ROMS=<jsmolka gba-tests>` `sh tools/tests/run_gba_test.sh` (`arm`, `thumb`, `memory`, `bios`, `nes`, `unsafe`, `saves`); `tools/tests/gba/gbatest.cpp` `<rom>` `<seconds>` `[out.ppm]` `[keys]` runs a game headless

(GBA_SHOTS, GBA_AUDIO, GBA_SAVE, GBA_LOAD, GBA_REGS; built with -DGBA_DEBUG, GBA_WATCH=<addr> prints every write there). **NES core** (user/nes/nes.h, nes/libnes.a): nes::Machine — the same shape (load an iNES / NES 2.0 file, runFrame → fb 256×240, setButtons (nes::BTN_*), setAudioRate, audioRead, setSaveRam / sram (8 KB) / battery / sramDirty, setPal). The 6502 (official

- stable unofficial opcodes, from the generated table nes_optable.inc, stepped an instruction at a time with the PPU and APU brought up to each access; the PPU (nes_ppu.cpp) renders a line at dot 1 from the loopy v / t registers, with the sprite 0 hit dot, vblank / NMI and the A12 edges the MMC3 counts; the APU (nes_apu.cpp: 2 pulses, triangle, noise, DMC, the frame counter and its IRQ) mixes through the nonlinear tables nes_mix.inc. Mappers 0, 1, 2, 3, 4, 7, 66; NTSC and PAL (312 lines, 3.2 dots per CPU cycle, the PAL APU tables; load reads it from the header, nesemu also from the file name). Used by nesemu and gamelib. **Host test:** NES_TEST_ROMS=<christopherpow nes-test-roms> sh tools/tests/run_nes_test.sh (nestest against its trace, blargg all_instrs / instr_timing, apu_test 1-8, mmc3_test 1-3 and 5); tools/tests/nes/nestest.cpp run <rom> <seconds> [out.ppm] [keys] runs a game headless (NES_PAL=1, NES_AUDIO=<file>). **Super Nintendo core** (user/snes/snes.h, snes/libsnes.a): snes::Machine — the same shape (load a .sfc / .smc, a copier header skipped; runFrame → fb 256 × height (224, 239 with overscan); setButtons (snes::BTN_*), the pad's 12 buttons as the joystick word; setAudioRate, audioRead; setSaveRam / sram / sramSize / sramDirty; setPal; title, hirom, chip). snes.cpp: LoROM / HiROM (the header that scores best), the bus as 4 KB pages (direct reads, the access speed of each page, FastROM), the 5A22's registers, DMA and HDMA (direct / indirect, the 8 transfer modes), a line = 1364 master clocks with its events (HDMA start, DRAM refresh, the H/V IRQ, the line drawn + HDMA) run between the instructions. snes_cpu.cpp: the 65C816 (every opcode and mode, the emulation-mode wraps, decimal mode, block moves). snes_ppu.cpp: modes 0-7 a line at a time (tile rows decoded 8 pixels at a time, offset-per-tile, mosaic, direct colour, Mode 7 + EXTBG, 32 sprites a line, the windows, colour math, brightness; modes 5 / 6 at half their width). snes_apu.cpp: the SPC700 with its IPL ROM and timers, the S-DSP (BRR, gaussian interpolation — snes_gauss.inc, a generated gaussian kernel —, ADSR / GAIN, noise, pitch modulation, echo + FIR), resampled from 32 kHz; the sound unit catches up with the CPU when the CPU touches its ports and every 16 lines. No enhancement chip (load refuses them). Used by snesemu and gamelib. **Host test:** SNES_TEST_ROMS=<PeterLemon SNES> sh tools/tests/run_snes_test.sh (the CPU, SPC700 and PPU test ROMs against their reference pictures; SNES_CPUTEST= gilyon's cputest-full.sfc, 649 tests); tools/tests/snes/snестest.cpp <rom> <seconds> [out.ppm] [keys] runs a game headless (SNES_SHOTS, SNES_AUDIO). **Nintendo 64** (user/n64/, in progress): n64_cpu.cpp the R4300i interpreter (MIPS III, COP0 with the TLB / exceptions / Count-Compare, COP1), n64_bus.cpp the RCP interfaces (MI, VI, AI, PI DMA as ares does it, SI + PIF, SP / DP registers) and the boot (the IPL3's work done directly, CIC 6101-6106), n64_gfx.cpp the graphics tasks at a high level (F3DEX2: matrices, lit vertices, triangles; the RDP: TMEM loads, tiles, the texture formats decoded into a cache, the combiner evaluated per vertex, blending / depth — the decal z mode drawn $2 \cdot 10^{-4}$ nearer in NDC depth, against z-fighting —, fill and texture rectangles; the rectangles into an off-screen colour image drawn by the CPU straight into RDRAM, and the renderer's framebuffers written back into RDRAM from the host's copy of the last frame (Machine::fbSnapshot, called by n64emu / NintendoEMU after each frame shown) when a game loads a texture from one — Ocarina of Time's pause background, copied into the z-buffer, its 8-bit coverage copy written as full so that the CPU anti-aliasing filter leaves it alone: the menu opens in ~1 s instead of ~4) into a GFrame of clip-space vertices + batches in the kapi v54 layout; n64_audio.cpp the audio tasks at a high level (the "nead" microcode of Zelda OoT / MM, recognised by its data: VADPCM, resampling, envelope mixer, interleave on a 4 KB DMEM image;

other audio microcodes: silent) and the AI output (each DMA's frames resampled into a ring, `audioRead`). `n64emu` draws the frames with `kapi_gpu_render` (the machine on an app core, lockstep). **Host test:** `N64_TEST_ROMS=<PeterLemon N64> sh tools/tests/run_n64_test.sh` (89 of the 94 CPU test ROMs match their pictures); `tools/tests/n64/n64test.cpp <rom> <frames> [out.ppm]` runs a game headless (`N64_GFX=out.ppm`: the last graphics frame drawn by the BASIC 3D's software renderer; `N64_STATE`, `N64_THREADS` (libultra's threads found in RDRAM), `N64_WHO`, `N64_MEM`, `N64_WAV=out.wav`: the sound, 32 kHz stereo). **GameCube** (`user/gc/`, in progress): `gc_cpu.cpp` the Gekko (PowerPC 750CL) interpreter: integer, branch, the SPRs (BATs, HIDs, GQRs, the locked-cache DMA, timebase, decremter), the exceptions, the FPU (exact single $\leftrightarrow$ double conversions, 25-bit multiplicands), the paired singles and the quantized loads / stores; `gc_mem.cpp` MEM1 (big-endian), the locked cache, the BAT translation (a map of 128 KB blocks); `gc_hw.cpp` PI, VI (lines, display interrupts, the YUV framebuffer), SI pads, EXI (RTC / SRAM), DI (disc reads), AI, the DSP interface (audio / ARAM DMA); `gc_gx.cpp` the CP / PE and the GX FIFO (linked mode: read as the CPU writes it; the commands, display lists, vertex sizes from VCD / VAT, tokens, "draw done"); `gc_gxdraw.cpp` the drawing at a high level: vertices decoded and transformed (XF matrices, projection, viewport), lit, texgens, the TEV per vertex (texel $\times c1 + c2$), the GX texture formats (TLUTs in TMem; their mipmaps for a GPU) $\rightarrow$ a `GFrame` in the `kapi v54` layout, ended by the EFB $\rightarrow$ XFB copy — or, with a GPU backend (`Machine::gpu`), the draws as the game gave them (`gc_gxgpu.cpp`, below); `gc_dsp.cpp` the DSP at a high level (as Dolphin's HLE): the ROM's boot mails, the microcode told by its CRC (AX, the "Zelda" one and its variants' flags, the memory card's), the mail queue with its interrupts (a mail's IRQ raised when the previous one is read, none seen while the DSP is halted), the task switch (DSP_YIELD, a new microcode, DSP_RESUME); AX's protocol acknowledged (silent), the Zelda protocol (command lists acked with sync mails, the frames rendered as the CPU says the voices are ready, DSP_FRAME_END) with its **audio renderer** `gc_zelda.cpp` (Dolphin's `ZeldaAudioRenderer`: the 0xC0-word voice blocks, AFC ADPCM and PCM8 / PCM16 from ARAM or MRAM, 4-tap resampling, volume ramps, Dolby, the reverbs, 0x50-sample frames into the game's buffers — a game on this microcode, e.g. The Wind Waker, waits for its streams to play: without it, a black screen after the intro); `gc_card.cpp` the **memory card** (an EXI device: the Nintendo commands 00 / 52 / 81 / 83 / 85 / 89 / F1 / F2 / F4, DMA with the card's delays, EXIINT, the EXT bit; a blank card formatted as the SDK's `CARDFormat` would, its serial from this console's SRAM, the SRAM's flash ID taken back from the card; the flash is the front end's: `cardInsert`, Dolphin's `.raw` layout, `card[s].dirty` after a write); `gc_boot.cpp` the IPL's state, `.dol` loading, discs booted by running their apploader on the CPU. The **sound out**: the audio DMA's 32-byte blocks (8 frames, right channel first, big-endian; 32 or 48 kHz by AICR's AIDFR, 32 kHz as the IPL leaves it) resampled to the front end's rate (`setAudioRate`) into a ring that `audioRead` empties. `runFrame` runs **one field** (the lines 1..313 / 314..625 in PAL): a front end calling it 50 / 60 times a second plays in real time (it ran a whole frame before: twice too fast). `gcemu` reads a disc image on demand (`kapi_seek`, the main thread serving the app core). The **write-gather pipe** (0x0C008000) keeps its bytes (`gather`) and sends them to the GX FIFO 32 at a time, as the hardware (`gatherFlush`: the burst at the PI's write pointer, then the commands run); the JIT appends a store to it inline (a call only at each burst). **The JIT** (`gc_jit.cpp`, AArch64 hosts): `Machine::jitEnable()` (code memory from the host's `gc::codeAlloc` hook: `kapi_code_alloc` (ABI v58) on Onyx, `mmap` RWX in the host test), then `run()` goes through `jitRun()`: the PowerPC code is translated a **block** at a time (to its branch, ≤ 64 instructions, within a 4 KB page) by a small built-in AArch64 assembler. The guest registers live in the `Machine` (`x19 = it`, `x20 = MEM1`, `x21 = the helpers + a 64K-entry direct-mapped table of blocks`, `w22 = MEM1's size`); **within a block** the GPRs, CR, XER, LR and CTR it uses are cached in host registers (`x9–x15`, `x18`: loaded at first use, the dirty ones written back at the exits, before the

interpreter, and around a slow-path call); **LR, CR, r3 and r0** — the most used — **live in x24, x27, x28, x29 across the blocks** (`SRA_G` / `SRA_H`: callee-saved, the helpers keep them; loaded by the enter stub, stored by the exit stub and around the interpreter's calls, `lmw` loading them straight: no load at a block's first use, no store at its exits); the integer unit, CR logic, rotates, shifts, compares, `divw` / `divwu`, the branches (CTR / CR conditions, LR), `mftb`, the loads / stores, `lmw` / `stmw` and `dcbz` (MEM1 in one go) are native (a load / store whose address maps MEM1 through the OS's standard BATs, or in real mode, reads the host memory directly and byte-swaps -- with the BATs, the 0x80000000 mirror checked by one EOR and a compare, the value loaded straight into the destination's register or stored from the source's, the uncached 0xC0000000 one tried first on the cold path; a D-form access through a base register the translation finds in MEM1 (and not yet written in the block) makes that register's host pointer (`basePtr`: checked there -- not MEM1's then: the instruction by the interpreter, and out of the block) and the block's later accesses through it are one load / store with the displacement (`ptrAccess`; MEM1 has a guard zone of `MEM1_GUARD` bytes on each side for the displacements past its ends); the rest calls `read32...` with the cycle count exact), every other instruction (the FPU, the paired singles for now) calls the interpreter's `exec`. The FPU and the paired singles are native too, on NEON: `ps[32][2]` keeps both halves of an FPR side by side, so an FPR is one q register, cached like the GPRs (q8–q31, lane 0 = ps0); the paired singles compute both halves at once (`FADD` / `FMLA .2D`, by-element for `muls0` / `madds1`, `FCVTN` + `FCVTL` for the single rounding, `ZIP` / `EXT` for the merges, `FCMGE` + `BSL` for `ps_sel`); the plain FPU works on lane 0 (`fadd...` `fmadd`, `fres` / `frsqtr` (`ps_res` / `ps_rsqtr` on both), the 25-bit multiplicand — skipped when the operand is known to hold a single exactly —, `fsel`, `fcmp`, `frsp`, `fctiwz`, the moves), `lfs` / `lfd` / `stfs` / `stfd`, `psq_l` / `psq_st` and their indexed forms, **specialised for the GQR's value found at translation** (a float type: one 8-byte access for the pair; `u8` / `u16` / `s8` / `s16` with a scale: both elements in one access, converted and scaled, or rounded to single, scaled, truncated and clamped), the GQR checked at run time (another value: the interpreter); a NaN result re-runs the instruction in the interpreter (from the cache's state at that branch), `FPRF` is set lazily (the source FPR noted at translation, `fprfVal` / `fprfPending` written only when needed). The cycles are a countdown in x26 (to `jitEnd`, = `jitUntil` when set; written to `cycles` when leaving and before a helper that reads them, resynced after one that may change `jitUntil`). The rare paths (a slow memory access, a NaN, a conversion's odd case, the interpreter leaving, an exit's stub) are emitted apart, each with the cache's state at its branch: the code buffer is in 1 MB chunks, the blocks' main code from a chunk's start up, their rare paths from its end down (a `b.cond`'s reach) -- the main code dense in the I-cache (--`pmu=08,02,05,01` counts the TLB and L1I refills: the L1I ones were ~20 a thousand instructions, 17.5 with the split; the TLBs, with Onyx's 64 KB pages, ~0.3). In The Wind Waker's game (Outset, ~96k vertices a frame) -- `pmu=08,01,52,53` gave 11.4 L1I refills and 3.2 L2 read refills a thousand instructions (0.1 write ones) — about as many as the L1D refills: the machine's data comes from the RAM, its IPC ~0.84; with --`nodraw` 37 → 30 M cycles a field for the same instructions (IPC 1.02): the display's copies on core 0 (through the L2 the four cores share) cost it ~18 %. A block's exit to a known address is **linked**: once that block is translated the exit's branch is patched to jump straight to it (back to its stub when that block is dropped), while `cycles < jitUntil` (the next event / the decrementer; `piUpdate` and `decWrite` zero it to come back); an exit to a register (`blr`, `bctr`) goes through the table; `jitRun` (C) takes the interrupts, translates what is missing. **Calls and returns are the host's own**: a call (`bl`, `bcctr1`) pushes a pair {its landing, the guest's return address} on the host's stack (`stp x1, x24, [sp, #-16]!`) and calls the target's block with `bl` (linked like an exit) / `blr` (`bcctr1`: the block looked up in the table there); a return (`blr`, `bclr`) pops the pair and, LR being its address, is a `ret` to the landing — which the core's return stack predicts —, else the stack is emptied and the dispatcher goes on (`retTo`, `callTo`, `callReg`); the landing, in the caller's block, is

a linked exit to the instruction after the call (the host registers are the callee's there: the cache dropped). The stack holds a sentinel at `jitSpBase` (no LR is 1); it is emptied when the JIT leaves (the exit stub), at a mismatch, and past 8 KB (`jitSpLimit`: calls that never return, a return through `bctr`); a landing stays valid while the JIT runs (a dropped block's code stays in the buffer until `flushAll`, which runs from `jitRun` only). `b1 $+4` (the pc read) pushes nothing. The Wind Waker: the branch mispredictions 5.6 -> 2.0 a thousand instructions, 37 -> 42 fields/s (its hottest loop, a list searched through two function pointers, went through the dispatcher four times a node). `gctest calltest` checks it against the interpreter. Blocks are keyed by address + MSR IR / DR; `icbi`, the DVD / ARAM / locked-cache DMAs drop the blocks of the 4 KB pages written; a BAT change, H1D0's ICFI or a reset drop everything. `b .` jumps to the next event (idle), and so does a **polling loop** (a block of loads, compares, masks branching back to its start, each register it reads set earlier in the same pass or not by it at all: nothing changes until an interrupt / the hardware — the VBlank waits, the DSP's mail). A compare (or a record form) right before a conditional branch on its field's LT / GT / EQ: the branch tests the host's flags, and the CR field is made only on the paths where it may still be read (`crSet`, `crDead`: the guest code of the block's 4 KB page scanned from each path -- set again before a read: not made; a call (`b1`) sets CR0 again as far as the caller knows — the ABI's volatile field: the callee may set it, the caller does not read it after without setting it again —; a return, leaving the page, a CR logic op on it: made) -- an exception that does not come back then leaves the older field, which only a crash report shows. `gcemu`: Game ▶ *Interpreter (no JIT)* / `--interp` to compare. Tested by `run_gc_test.sh` under `qemu-aarch64` (`GC_JIT=1`): `cptest` / `pstest` / `hwtest` / `gxtest` identical to the interpreter's, `gctest fuzz` (random sequences of FPU / paired-single / load-store / integer / branch instructions from random states with NaNs, infinities, denormals, run by both, every register, the FPSCR and the memory compared: it found a cache bug and an interpreter bug — `srawi` / `sraw` with `rA = rS` took CA from the result; `GC_FUZZC=1` their data through the uncached mirror, `GC_FUZZSTART=n` from the n-th, `GC_FUZZDUMP=<f>` each one's code, `GC_FUZZPROG=<f> + GC_FUZZLEN=n` a failing one cut down, `GC_FUZZG="3 11"` those GPRs, CR and XER printed, `GC_FUZZBLOCK=<pc>` the block there's code; `gctest calltest`: a program of calls built there (fib, a chain of 1500 calls, calls through a table, a tail call, `b1 $+4`, a return to LR + 4, returns through `bctr`) run by both, twice (the blocks kept the second time), every register and the stack compared; `GC_FUZZ2=1` each one run a second time with the JIT's blocks kept and the data through the other mirror (the blocks' base pointers meet another value); the CR is not compared after an exception that stopped the run -- see `crDead`), `tools/tests/gc/bench.c` (sort, CRC, copies, calls) against `qemu-ppc`'s result: ~40× the interpreter's speed (integer), ~10× (float); `GC_PROFILE=1` prints the host instructions per guest instruction, `GC_DUMP=prefix` the hottest blocks' code (for `aarch64-linux-gnu-objdump -b binary -m aarch64`); `gctest jitsize <jitprof.bin>` translates again the hot blocks of a Pi's profile (`gcemu --jitprof`) and prints their host instructions a guest one weighted by the runs counted there (6.56 → 5.85 with the base pointers, the hot / cold chunks and the kept registers), `GC_JITDUMP=<pc>` one's code. **The x86-64 JIT** (`gc_jit_x64.cpp`, NintendoEMU / x86-64 Linux hosts; `gc_jit.cpp` is the AArch64 one): the same design (blocks, links, the dispatcher's table, idle loops, the rare paths after the block, the cycles as a countdown) with a small x86-64 assembler (REX / ModRM / SIB, SSE2, VEX for FMA3): the guest GPRs cached in `rbx`, `rbp`, `rsi`, `rdi`, `r12`, `r10`, `r11`, the FPRs in `xmm6–15` (a paired single = one `xmm`, both halves), the constants in the code buffer (RIP-relative); the helpers take only the `Machine *` (their arguments in `jitArg[]`: the Windows and System V ABIs alike). The same fuzz test (`gctest fuzz`) checks it against the interpreter. Two fixes found with it and shared: H1D0's ICFI / DCFI read back 0 (a game setting ICFI flushed every block), and each AI sample is an event while the AI plays (`__AI_SRC_INIT` polls its counter). **The GX on a GPU with shaders** (`gc_gxgpu.cpp`): a front end sets `Machine::gpu` (a `GxGpu`: `draw`, `copy`); each

primitive then reaches it as the game gave it — its vertices in model space with their matrix indices (`GxVertex`), its triangles / lines / points as indices — with a `GxState`: the XF's state (colour channels, texgens, projection) and the BP's (the 16 TEV stages, swap tables, konst colours, alpha test, z texture, fog, indirect texturing, depth / blending / culling / scissor, the maps' textures) laid out as a std140 uniform block, rebuilt when a register changed (`gxDirty`; the XF memory — matrices, lights — has its own `xfSerial`). The EFB copies go to it too: to the XFB (the picture), or to a texture — kept by the GPU, not written to MEM1: a slot (`efbCopies`) remembers the address and a hash of MEM1 there, so a texture read from that address (MEM1 not written since) is the copy. NintendoEMU's backend is `pc/NintendoEMU/core/gxgl.cpp` (below); without a backend (the Pi today) nothing changes. **Host test** (`sh tools/tests/run_gc_test.sh`, needs gcc-powerpc-linux-gnu + qemu-user): `tools/tests/gc/cputest.c` compiled once runs under `qemu-ppc -cpu 750` and in the interpreter (10485 result words identical), `pstest.S` the paired singles against the manual, `hwtest.c` and `gxtest.c` bare-metal programs (built with `tools/gc/elf2dol.py`): the VI's picture and interrupts, the FIFO / PE, a textured quad and a shaded triangle drawn by the GX path. **Doom** (`user/doom/`): `doomgeneric` (`third_party/doomgeneric`, GPL-2.0, only the portable sources; `ONYX.md` lists the three `#ifdef ONYX` changes) built against **newlib** like the `/bin/libc` tools (`../libc/crt0libc.S` + `onyx_syscalls.c`, `main (void) + kapi_get_args`), by `user/doom/Makefile` (called from `user/Makefile`) into `user/doom.elf`. `doom_onyx.c`: the `DG_*` platform functions — the window canvas is `DG_ScreenBuffer` (640 × 400, no copy), full screen at 4:3, keys from `kapi_key_held` + modifiers + key events (Tab, F-keys...), `gamepad.h`, `rename/mkdir` on the `kapi`; `doom_wtk.cpp`: window chrome + menu (wtk from C); **The engine runs on an app core** when one is free: `main` starts `doomgeneric_Tick` there after `doomgeneric_Create`; the main thread keeps the window, the input (a key queue), the pictures (a triple buffer: the engine swaps `DG_ScreenBuffer` between three slots), the sound output and the music. The engine's syscalls (malloc's `sbrk`, `stdio`, files, saves, `exit`) run on the main thread through the **libc RPC** (`libc/onyx_syscalls.c`: `onyx_rpc_enable`, `onyx_rpc3`, `onyx_rpc_serve` — any newlib program can use it for code on an app core; the main thread must then not use malloc meanwhile); `doom_sound.c`: `DG_sound_module` (16-channel mixer of the DS* lumps, run by the engine into a PCM ring that the main thread moves into `kapi_sound_write`) and `DG_music_module` (MUS and MIDI turned into one timed event list, played on the 16 kernel FM voices with the GENMIDI OPL patches converted to `kapi_fm_instrument`). **Host test**: `sh tools/tests/run_doom_test.sh` (headless on a virtual clock: frames, sound effects, FM notes). **Graphing calculator expressions** (`user/Apps/graphcalc/expr.h`): `gc::Parser::compile (src, program) → an RPN gc::Program (eval (x)), gc::fmt`; the maths of the BASIC core (`basic/basnum.h`). An app computing in `double` builds with FP: in `user/Makefile`, `graphcalc.elf`: `CXXFLAGS := $(CXXFLAGS_FP)`. Host test: `sh tools/tests/run_graphcalc_test.sh`. **HTTPS from a wtk app**: wtk apps are freestanding; do the TLS work in a newlib console tool and spawn it with pipes (`kapi_pipe`, `kapi_spawn`, write the request, `kapi_stream_eof`, poll `kapi_stream_read_nb` / `kapi_proc_done` from `Root::onTick`). Example: `Lisa + /bin/groq` (`user/Apps/lisa`, `user/bin/groq.cpp`). **Wi-Fi scan (ABI v45)**: `kapi_wlan_scan (ap, max)` fills struct `kapi_wlan_ap` entries (ssid, bssid, security `WLAN_SEC_*`, channel, freq, level dBm, connected), strongest first; it blocks ~3 s. Examples: `user/bin/wifiscan.c`, `wpaconf` (Scan button + Combobox). An app that **moves or renames** files should call `shelf_moved (from, to)` (`#include "shelfmsg.h"`, IPC to the `shelf` service: the dock's switcher) so the shelf's references follow. **WPF-style controls (P5)** — all in `wtk/wtk.h`, see `user/Apps/widgets` for each in use: `RadioButton (l, t, w, h, text, group, checked, cb)` — exclusive per group among its siblings (`wk_radio_checked (parent, group)`); `GroupBox (l, t, w, h, title)` — a titled frame, add controls as its children; `ToggleSwitch (... , text, on, cb)`; `NumericUpDown (... , min, max, value, step, cb)` — arrows, wheel, Up/Down, typed digits;

`ListBox (... , onSelect, onActivate)` — `add, clear, item (i), sel, setSel`; `DataGrid (l, t, w, h)` (`gtk/datagrid.h`) — a read-only table of rows and columns, virtual (a grid of any length costs what it shows): `setColumns (n), setColumn (c, title, width, GRID_LEFT / GRID_RIGHT / GRID_CENTRE), column (c), autoSize (c, minW, maxW)` (the title's and the first rows' texts), `setRows (n), sel, setSel (r), ensureVisible, rowAt`; the app gives each cell's text (`cellText (grid, row, col, buf, cap)`) or draws it itself (`cellDraw (grid, cv, row, col, x, y, w, h, ink, selected)`: `cv` clipped to the cell; false → the text is drawn); a click on a title fires `onSort` (`clickedCol`: the app orders its rows and sets `sortCol / sortDesc`, the grid shows the arrow; `sortable = false`: plain titles), a title's right edge drags the column's width; `onSelect, onActivate` (a double click, Enter), `onContext` (a right click: `ctxRow` — -1 below the rows —, `ctxX, ctxY`); stripes, the row under the pointer tinted, both scroll bars (Shift + the wheel, Left / Right: sideways), `emptyText, user` (the app's). Used by Cardfile (its list, its design's fields); `TreeView (... , onSelect, onActivate)` — `add (parent, label) → id, expand, sel, label (id), setUserData`; `Calendar (l, t, y, m, d, cb)` (size `CAL_W×CAL_H`) and `DatePicker (... , y, m, d, cb)` (format (`buf`) → `YYYY-MM-DD`); `ImageBox (... , IMG_FIT / IMG_FILL / IMG_NONE)` — `load (path)` (any `imgload` format) or `setPixels`; `wk_color_dialog (&color, title)` — RGB sliders + palette + preview, true = OK. `wk_file_open / wk_file_save / wk_folder_open (out, cap, startDir)` (a folder, no file name) — the file dialog; `..` at a volume's root lists the **volumes** that are mounted (`SD:`, `SD1: ... SD3:` — the SD card's partitions, `USB:...`). Paths may start with any volume (`SD1:/roms/x.iso`); `kapi_fsize` is clamped to 4 GB – 1, `kapi_fsize64` (ABI v59) gives an exFAT file's real size; `kapi_rename` fails across volumes (copy + remove instead). A `Textbox` holds **63 bytes** unless you raise its `maxLen` (up to `Textbox::TEXT_CAP - 1, 511`): IRC's chat line takes 400. `Combobox (l, t, w, h, text, onEnter, onPick)` — an editable `Textbox` with a drop-down list of suggestions (`addOption, clearOptions, pick (i)`; arrow click or Down / Up; `onPick` fires with `picked = the index`). Used by the File Viewer's Connect dialog. `Dropdown (l, t, w, h, options, n, initial, cb)` — the non-editable sibling, same look (field + drop button): `sel, setOptions`; Up / Down change the selection, Enter / Space open the list, Esc closes it; `cb` fires when the selection changes. The option strings are not copied (they must outlive the widget). **Keys with modifiers**: Ctrl / Shift + arrows, Home, End, Page Up / Down arrive as the plain `KEY_*` code; test `kapi_get_modifiers () & MOD_CTRL / MOD_SHIFT` (e.g. the FM Tracker's Ctrl+Up / Down transpose, the text widgets' Shift selection). Inside a key handler, `kapi_get_modifiers` returns the modifiers held **when that key was typed** (the kernel stores them in the key event, taken from the xterm `ESC[1;<m>X` form, which `vncd` also sends), not the live state. F1–F12 arrive as `KEY_F1 .. KEY_F12` (0x110..0x11B). Text widgets (`Textbox, Textarea, RichTextBox`) accept the printable Latin-1 range too (é è à ç ù... = 0xA0–0xFF, as the keymaps produce them). The euro sign (AltGr+E, ...) arrives as **0x80**, Windows-1252's code for it (Latin-1 has none); `wtk`'s font draws it there (`tools/fonts/gen_nssans.py, EXTRA`); the text widgets do not take it yet — the Spreadsheet does (`latin1_cp → U+20AC`). With a **text face** installed (below) `Textbox` and `Textarea` hold **UTF-8** instead: a typed Latin-1 key is stored as its UTF-8 (0x80 as U+20AC, the euro), the caret moves and deletes whole characters. **Text selection**: `Textarea` and `RichTextBox` select with Shift + navigation keys, a mouse drag, Shift+click and ^A; typing replaces the selection. `Textarea` has `hasSelection, selStart / selEnd, selectedText, deleteSelection, selectAll, copy / cut / paste` (system clipboard; ^C / ^X / ^V work by themselves when the app's menu does not take them). **Tooltips**: `set widget->tip = "text"`; the Root shows it after the pointer rests ~0.6 s. (No RTTI: `Widget::asRadio ()` identifies radio buttons.) `wtk::Root::onTick ()` (virtual) runs once per event-loop iteration — poll a mailbox, a spawned process or a timer there. `ask.h: ask_begin (title, msg, yes, no)` opens the system Yes / No window (`apps/ask`) without blocking; `ask_poll (h)` returns -1 while open, then 1 / 0. `ask_text_begin (title, msg, ok, cancel,`

`text`) asks for a line of text the same way (for an app that never has the keyboard: the dock); `ask_text_poll (out, cap)` returns -1, then 1 (the text in `out`) / 0.

Text faces — anti-aliased, proportional text in every widget (`gtk/text.h`). By default `gtk` draws with its bitmap fonts (8 × 16 cells); an app may install a `TextFace` instead and every text path of `gtk` goes through it: `Canvas::text`, `wk_text_l / wk_text_c / wk_text_w`, the line height widgets lay out with (`wk_fh ()` = the face's height (); `wk_fw ()` = a digit's width), and the widths they compute — a `NumericUpDown`'s right alignment, tooltips, `Icon` labels, the `DataGrid`'s cells (cut with "..." by measure), `Calendar`, `PopupMenu`, the text boxes' carets. `Textbox` and `Textarea` place the caret, the clicks and the selection by the glyphs' real widths (a `Textarea` then scrolls sideways by pixels: `leftPx`). Without a face, nothing changes — byte for byte (the screenshots stay identical). Not through the face: the window's **frame** (its title keeps the desktop's bitmap font, the same on every window), `RichTextBox` (its own styled bitmap glyphs), and the explicit bitmap calls `Canvas::drawFont / gtk::draw_text`.

- The interface: `struct TextFace { virtual int height (); virtual int ascent (); virtual int width (const char *utf8, int style); virtual void draw (Canvas &cv, int x, int yTop, const char *utf8, unsigned color, int style); virtual int widthN (utf8, n, style); }` — styles 0 regular, 1 italic, 2 bold, 3 bold italic; text in UTF-8 (a stray byte is read as Latin-1); `draw` blends over the canvas, the line's top at `yTop`. `wk_set_textface (f)` installs it (0: back to the bitmap fonts), `wk_textface ()` returns it. Install it **before building the widgets** (some size themselves from `wk_fh ()` when made: a `DataGrid`'s rows, the dialogs).
- Measure and draw through it (they fall back to the bitmap fonts): `wk_tw (s, style)`, `wk_tw_n (s, n, style)` (a prefix: a caret's x), `wk_tpos (s, n, x, style)` (the character boundary nearest x: a click), `wk_text (cv, x, y, s, c, style)` (top-left), `wk_text_clip (... , cx, cy, cw, ch)` (clipped to a box), `wk_text_fit (s, w, out, cap)` (cut to w px with "..."), `wk_bfw / wk_bfh` (the bitmap cell whatever the face). UTF-8 helpers: `wk_u8_len / _get / _next / _prev / _put`, `wk_u8_key (k, out)` (a typed key's UTF-8). `WkFaceScope sc (face)`; draws with another face until the end of the scope (a widget's captions, a display's large digits).
- **FreeType's face** (`user/ft/wtkface.h`, header-only, one translation unit; a **newlib** app linking `ft/libft.a`, as `Writer` — §5.1): `ft_wtk_install ("DejaVu Sans", 13)` at the start of `main` (before the `Root` and the widgets) makes an `FtTextFace` on `ft/fonts.h` (the card's TrueType families of `SD:/res/fonts / SD:/fonts`, anti-aliased, quarter-pixel positioned, kerned, a small width cache; bold / italic from the family's files or made) and installs it; false: no TrueType font (the bitmap fonts stay). `DejaVu Sans` at 13 px has a 16-px line, as the bitmap font's, so layouts keep their rows. More faces for large or small text: `FtTextFace *f = new FtTextFace; f->open ("DejaVu Sans", 24)`; (e.g. an `LcdDisplay`'s face). `ft_wtk_face ()`: the installed one. The face holds no `fnt::Font *` between two calls (`fnt::trim` is safe).
- **Every new app uses the FreeType face** (unless told otherwise): add it to `FT_APPS` in `user/Makefile` — the `newlib + ft/libft.a` rule the Control Panel, its applets (Theme, Panel, Display, Sound, Keyboard & Mouse, Gamepad, Wi-Fi, App Settings), the Game Library and Setup share (`FT_EXTRA_<app>`: libraries of its own) — and to the same list in `tools/tests/desktop_sim/shots.sh`'s `build`. Measure text in pixels (`wk_tw`, `wk_text_fit`), never in characters, and draw it through the face (`wk_text`, `canvas.text`), not `drawFont` (the bitmap fonts only).
- On the PC: `sh tools/tests/desktop_sim/studio.sh [out]` builds `gallery/studio.cpp` with the FreeType face and with the bitmap fonts, in the card's theme and in a dark palette, the Widget Showcase under the face (`gallery/ftwrap.cpp`), writes their pictures (default `/tmp/onyx_studio`), and runs `facetest.cpp` (the measures, the carets and clicks, UTF-8 editing).

Studio controls (for Koton's DAW, usable anywhere; drawn from the theme's colours — a light theme and a dark one alike —, anti-aliased with `wtk/vpaint.h`). All in `wtk/wtk.h` but the toolbar: `#include "wtk/toolbar.h"` yourself (Writer, the Spreadsheet and Cardfile have their own `ToolBar` / `ToolButton` next to using namespace `wtk`, so `wtk.h` leaves it out). See them in the Widget Showcase (`user/Apps/widgets`, its Studio group) and `gallery/studio.cpp`.

Widget	Make it	What it does
Knob (<code>wtk/knob.h</code>)	<code>Knob (l, t, w, h, min, max, value, onChange); setLabel ("Gain"), showValue, format (v, out, cap), setDefault (v), bipolar, arcColor, step, face (captions)</code>	A rotary control: a 270° track, the value's arc in the accent (from the start, or from 0 when <code>bipolar</code> : a pan), a cap with a pointer, the label and the value under it. Drag up / down (the range in 200 px; Shift: 1000 px), the wheel, a double click → the default; keys Up / Down / Left / Right, Page Up / Down, Home / End, Delete (the default). The dial is the width, less the captions' lines (about 24 ... 64 px). <code>setValue (v, fire)</code> , <code>setRange</code> , <code>valueText</code> .
VuMeter (<code>wtk/vumeter.h</code>)	<code>VuMeter (l, t, w, h, vertical = true, stereo = true); floorDb / topDb (-48 / +6), amberDb / redDb (-12 / -3), segPx (3; 0 = continuous), holdTicks, fallDb, peakFallDb, showPeak, showClip</code>	A level meter: segments on a dark well, green → amber → red along a dB scale, the peak held then falling, a clip light (a level over 0 dBFS; a click clears it). <code>setQ16 (1, r)</code> — linear, 65536 = 0 dBFS —, <code>setCdb (1, r)</code> (1/100 dB), <code>set (float l, float r)</code> in an FP-enabled unit only (wtk itself is integer-only). Call it at the UI's rate, silence included (the falls are timed by <code>kapi_get_ticks</code>); it repaints only when a lit segment or a peak moves. <code>wk_q16_to_cdb (v)</code> .
SegmentedControl (<code>wtk/segmented.h</code>)	<code>SegmentedControl (l, t, w, h, labels, n,</code>	Mutually exclusive segments drawn as one

Widget	Make it	What it does
	<code>selected, onChange); equalWidths (false: by the texts), setLabels, setEnabled (i, on)</code>	pill, the chosen one in the accent (bold). A click, the wheel, Left / Right. <code>selected, select (i, fire), label (i), count ().</code> Labels copied (12 × 31 chars).
ToolBar + ToolButton (<code>wtb/toolbar.h</code>)	<code>ToolBar (l, t, w, h = 34): add (w, gap), addRight (w, gap) (anchored right), sep (), space (px), line, bg; ToolButton (w, h, tip, onClick) then ->setGlyph (WKT_PLAY), ->setIcon (fn, id) (the app's drawer), ->setText ("Loop"), ->setToggle (true, on), ->setSplit (arrowCb), ->fitWidth (); filled, raised, onColor, iconColor</code>	A strip of small buttons: an icon, a label beside it or alone, a toggle (on: the accent's tint, or filled — a play button), a split arrow (a palette, a menu), a tooltip. Flat until pointed (raised : always a face). The icons: <code>WKT_NEW</code> <code>OPEN SAVE UNDO REDO CUT</code> <code>COPY PASTE PLAY PAUSE</code> <code>STOP RECORD TO_START</code> <code>TO_END REWIND FORWARD</code> <code>LOOP METRONOME PLUS</code> <code>MINUS SEARCH MIXER</code> <code>SPARK GEAR</code> , drawn at any size by <code>wk_tool_glyph</code> (<code>cv, kind, x, y, size,</code> <code>ink</code>). Generalised from Writer's (which keeps its own).
LcdDisplay (<code>wtb/lcd.h</code>)	<code>LcdDisplay (l, t, w, h, text, caption); setText, setCaption, setSub (repainted only on a change), face / smallFace, scale, ink, centred</code>	A time / position display: a sunken well (dark in a dark theme, the accent's pale tint in a light one), large digits in the accent — the <code>face</code> given (an <code>FtTextFace</code> at 24 px), else the bitmap font scaled as large as fits —, a small caption over a second line at its right ("BAR.BEAT.16" / "0:14.83").

The look: the modernised CDE (kapi v64). Every control, and every window's frame, is drawn by code from a few theme colours — no bitmap ([docs/gui-redesign/README.md](https://docs.gtk.org/gtk4/gtk4-re-design/README.md)).

- **The palette** (`wtb/theme.h`): `C_BG` (an app's background: the face), `C_FACE`, `C_FACE_HI`, `C_FACE_DN`, `C_BORDER`, `C_TEXT` (on the face), `C_ACCENT` (focus, selection, checks), `C_DIS`, `C_FIELD` (a text field's, a list's background), `C_FIELD_TEXT`, `C_SEL_TEXT` (on the accent), `C_FRAME_ACTIVE` / `C_FRAME_INACTIVE`

(the frames), `C_DOCK`, `C_BUTTON` / `C_BUTTON_TEXT` (a push button's, a drop-down's face: draw buttons with them), `C_MENUBAR`, `WK_OUTLINE`. They are **variables**, read once from `SD:/etc/theme.txt` by `wtk::init ()` (the Root's constructor calls it): use them in drawing code, never copy them into a `static const` or a global initialised at start-up (that runs before the theme is read). The file: `theme = Peach / Steel / Sage / Brick / Slate` (the window in front's frame; `active` = any colour instead), `inactive`, `window` (the content: the face; `face` still read), `button`, `field`, `menubar` (these three follow the window's colour when absent), `accent`, `outline` = none / dark / black, `dock` — the Control Panel's Theme applet writes it. As values: `WkTheme (WK_AUTO`: derived from the window's), `wk_theme_defaults`, `wk_theme_parse (text, t)`, `wk_theme_get (t)` (the palette in use), `wk_theme_set (t)` (the palette made from it: a preview), `wk_theme_write (t, out, cap)` (`theme.txt`'s text), `wk_theme_reload ()` (a new theme applied: the dock, the menu bar). Text: `C_TEXT` on the face, `C_FIELD_TEXT` on a field, `C_SEL_TEXT` on the accent, `C_BUTTON_TEXT` on a button, `wk_ink_on (bg)` on any colour. Content keeps its own colours (images, games' boards, a terminal's screen); the chrome around it follows the theme.

- **The painter** (`wtk/paint.h`, integer only): `wk_tone (c, level)` (a shade: 128 = `c`, 255 = white, 0 = black), `wk_mix`, `wk_rbox` (a rounded box with a vertical gradient, its corners anti-aliased: per-radius tables), `wk_rline` (its outline), `wk_framed` (the framed push button: a raised frame, a sunken well, the button in it — the gradients computed at its size), `wk_raised`, `wk_sunken` (a field), `wk_etch_h / _v / _box`, `wk_check_mark`, `wk_radio_mark`, `wk_switch_mark`, `wk_scroll_bar`, `wk_slider_mark`, `wk_progress_bar`, `wk_popup` (a floating panel, its corners keyed), `wk_hilite / wk_hilite_ink` (a selected row and its text), `wk_title_strip`, `wk_glyph` (`WKG_CHECK`, arrows, chevrons, close, minimise, maximise, restore, lock, gear, power, reload, home, history — a clock...), `wk_text_c / wk_text_l` (style 2 = bold). `tools/tests/desktop_sim/gallery/main.cpp` shows every control in every state; `gallery/studio.cpp` (`studio.sh`) the studio controls, with a FreeType face and without.
- **The frame:** `wk_decorate_window ()` (the Root calls it; an app drawing its own window calls it after `kapi_resize_window`) draws the title bar, the borders, the rounded corners (their outside see-through in the chrome's top byte), the title buttons — the window menu, minimise, maximise, close, at the kernel's `KAPI_FRAME_*` places — and the title in bold, in the active and the inactive frames' colours. Close and minimise are the kernel's; the window menu and maximise come to the app as `GUI_EVENT_WINCTL`, handled by the Root (the frame's state: `wk_window_state (WK_WIN_MENU | WK_WIN_RESIZABLE | WK_WIN_MAXIMISED)`, kept by the Root; an app drawing its own window without a Root gets the window-menu button greyed). An app with its own pointer handler answers `GUI_EVENT_WINCTL` itself: `KAPI_FRAME_MENU` → `root.windowMenu ()`, `KAPI_FRAME_MAXIMISE` → `root.maximise (!root.maximised ())` (the terminal does).
- **Resizable windows:** an app whose layout follows its window's size (anchored widgets, layout containers, or its own `layout ()`) calls `root.setResizable (true)`: its maximise button (and a double click on the title) fills the work area — between the menu bar and the dock — and restores it (`Root::maximise: kapi_resize_window2`, the canvas re-adopted, the frame redrawn), then `virtual void onResized ()`. Otherwise the button is greyed. The window menu (its button, top left): Restore / Maximise, Minimise, Move to *workspace* / On All Workspaces (the names: `SD:/etc/dock.ini`'s `desk` = lines), Close (`Root::windowMenu`). `root.fitWorkArea ()` (after the children are anchored): a window taller or wider than the work area is shrunk to it (a resizable one; else moved only) and moved into it — the Spreadsheet calls it at its start, so the dock does not cover its bottom.
- **The screen's size changing** (kernel v66, the Control Panel's Display applet): every window gets `GUI_EVENT_DISPLAY_RESIZE`; `wtk` calls `virtual void onDisplayResize (int w, int h)` at once

(an app placed by the screen's size — a borderless one: the dock — places itself again there), then ~0.3 s later fits a framed window by itself: maximised, to the new work area again (its restore size kept inside it); else moved into the work area, shrunk if it is resizable and too big. A borderless window is left to `onDisplayResize`. Outside wtk: handle `GUI_EVENT_DISPLAY_RESIZE` (`GUI_DISPLAY_W/H (value)`) in the pointer handler — the menu bar grows its canvas (`kapi_resize_window2`), the notifications move to the new top right.

- **PopupMenu** (`x, y`) (`wtk/dialog.h`): a pop-up menu — add (`label, id, enabled, hint`), `separator ()`, `run ()` → the id picked, -1 (a click elsewhere, Esc). A context menu.
- **See-through windows** (`WIN_FLAG_ALPHA`, borderless): the canvas's top byte is each pixel's transparency (0 opaque .. 255 see-through; a click on a wholly see-through pixel goes below). Clear to `0xFF000000`, then draw with `wk_paint_alpha (true)` so the anti-aliased edges over see-through pixels keep their colour (`wk_blend_px` for single pixels); `0xFE000000` (almost clear) still takes the clicks. Examples: `dock`, `menubar`, `agenda`. (Onyx Remote draws them over the other windows with the same transparency: `rdpd` sends them in 32 bits.)
- **A fixed window** (`WIN_FLAG_FIXED`, `kapi v69`): `Root (x, y, w, h, title, WIN_FLAG_FIXED)` — the user cannot move it, its frame has no buttons (wtk's `WK_WIN_FIXED`), and the kernel centres it again when the resolution changes (wtk does not fit it to the work area then). Setup's (`user/Apps/setup`: its pages in `main.cpp`, what it writes in `system.h`); with `kapi_screen_native` (the monitor's EDID size) and `kapi_set_timezone` (`v69` too).
- **Present what you draw**: the compositor and the remote desktop (`rdpd`: a window is sent again when its counter changes) see a canvas change at `kapi_present ()` — an app drawing in its own loop presents after drawing, and only when something changed (`eyes`: when a pupil moves).
- **Workspaces** (`kapi v65`): a window opens on the current desk; `kapi_desk (set, count)` shows desk `set` and/or sets how many there are (-1 / 0 keep them) → `KAPI_DESK_CUR (r)`, `KAPI_DESK_COUNT (r)`, `KAPI_DESK_GEN (r)` (bumped at every change: poll it to redraw a pager); `kapi_win_desk (id, n)` moves window `id` (0: yours) to desk `n` (-1: all; -2: ask). `kapi_list_windows` and `kapi_raise_app` see the current desk only; `kapi_win_list` sees all, `KAPI_WIN_OFFDESK` and `KAPI_WIN_DESK (state)` in their state. The dock is the pager (`dockconf.h`: the desks' number and names).
- **Control Panel applets** (`user/applet_proto.h`): any wtk app can be shown **inside** the Control Panel (`apps/control`) instead of in a window of its own. Started with `--applet <surface> <host pid>`, `wtk::Root`'s constructor sees it (`wk_applet ()`) and adopts the host's shared surface as its canvas (the pane's size, 700 × 470: lay out for it, or centre on `root.width`); the host copies it into its window at each present and sends the pointer and the keys over the mailboxes. `Root::run ()` does it all; an app with its own loop calls `wk_pump ()`, `wk_present ()` and `wk_quit ()` instead of `pump_events / kapi_present / should_exit` (they fall back to those alone). `wk_applet_send (AP_THEME)`: the host restarts the applet (a new theme applied). An applet needs no menu (the host has one) and never calls `kapi_create_window`. To list it, add a link file to `SD:/apps/control.app/applets/` (name, icon, target, text: the user guide §11) and give its `app.txt` category = Settings (the menu bar leaves those out). Examples: `theme`, `dockconf`, `soundconf`, `keyconf`, `config`, `padconf`, `wpaconf`.
- **Shared settings headers**: `dockconf.h` (the dock's drawers, launchers and workspaces: `SD:/etc/dock.ini`, read / written by the dock and the Panel applet; `DOCK_MSG_RELOAD` to the IPC service "dock"), `wallpaper.h` (the wallpaper's modes and their painter: `SD:/etc/wallpaper.ini`, used by `voronoy` and the Theme applet's preview; a pattern: `wp_paint` paints its gradient, `wp_grey_cover` lays the grey picture over the area as "cover" does — box-averaged when it shrinks, bilinear when it grows — and `wp_multiply` multiplies the colours by it; the shipped

patterns are made by `tools/gen_wallpapers.py`, `--preview` draws `screenshots/wallpapers.png`), `volume.h` (the master volume, `SD:/etc/sound.ini`).

- **Dropdown's** list opens **above** its box when it does not fit below and fits better there — every parent clips its children, so a drop-down near a window's bottom (or in a group box) keeps its list visible. Put the controls whose lists must pass a group box's edge in the window (the Theme applet's Desktop box does: `desk_add`).
- **Text on any background:** `wk_ink_for (bg)` is `C_TEXT` on the window's face (and on what is as light, or as dark), else black or white — the ink `Labels` and `Checkboxes` need on a program's own colour (the BASIC runtime's controls; a `Checkbox` picks it by itself).
- **Dialogs:** `wk_file_open` / `wk_file_save` / `wk_folder_open` (a double-click on a file picks it and confirms), `wk_color_dialog` (R / G / B sliders, a palette), `wk_messagebox`.
- **The desktop simulator** (`sh tools/tests/desktop_sim/run.sh`): a wtk app built for the PC against a stand-in kernel (`fakekapi.cpp`: files read from `sdcard/` — what the app saves goes to `/tmp/onyx_sim_writes`, never to the card —, a script of pointer / key / menu events, `dump` writes its window — frame and client — with its transparency), laid over the wallpaper by `compose.py`; `wmtest.cpp` checks the kernel's window manager on the PC. `shots.sh` makes the documentation's screenshots with it (below).

Menus. Put commands in the **system menu bar**, not in button rows. After creating the `Root`, build a `wtk::Menu` once and publish it:

```
static Menu menu;
menu.menu ("File");
menu.item ("Open...", "^O", WK_CTRL ('O'), onOpen); // Label, shortcut text, key, void()
// callback
menu.separator ();
menu.item ("Save", "^S", WK_CTRL ('S'), onSave);
menu.publish (); // kapi_set_menu (ABI v39)
```

The `menubar` app shows the menus while your window is the active app and sends the chosen item back (`GUI_EVENT_MENU`); `wtk::Menu` runs the callback, then invalidates the root. Every key goes through `Menu::shortcut` first (so item shortcuts work anywhere in the app); **Ctrl-Q** quits. Commands and shortcuts are ignored while a modal dialog is open. `^I`, `^H`, `^M` share their key codes with Tab, Backspace and Enter: `Menu::shortcut` only takes them while **Ctrl** is actually held, so the plain key still reaches the focused text box (still, prefer other letters).

Minimal skeleton (window with kernel-managed widgets):

```
#include "kapi.h"

static void on_ok (unsigned long sender, int ev, long value)
{
    (void) sender; (void) ev; (void) value;
    kapi_widget_set_text (g_label, "cliqué !");
}

static unsigned long g_label;

int main (void)
{
    /* Le canvas est mappé à 12 Go ; fb[y*w + x] = 0x00RRGGBB. */
    unsigned *fb = kapi_create_window (300, 200, "exemple");
```

```

g_label = kapi_add_label (10, 10, 200, 16, "prêt");
(void) kapi_add_button (10, 40, 80, 28, "OK", on_ok);

kapi_wait_for_exit (); /* pompe Les événements à ~60 fps jusqu'à fermeture */
return 0;
}

```

Key points:

- **kapi_create_window(w, h, title)** returns a pointer to the **canvas** (pixel buffer of 0x00RRGGBB, width w). The app draws directly into it (no per-pixel call). The variant **kapi_create_window_ex(x, y, w, h, title, flags)** is for explicit placement and **WIN_FLAG_BORDERLESS**. The client area is **at most the screen's size** (frame not counted; 1024 × 768 by default, width= / height= in `cmdline.txt`; before kernel v66, 1024 × 768 whatever the screen) — keep a window within 1000 × 700 or so, as Writer and Paint, so that it fits the default screen: over the limit (or out of memory) the call returns **0**. **Check it:** an app runs at EL1 with the kernel's identity mapping, so a null canvas is the kernel's own memory at address 0 — drawing into it overwrites the kernel and the whole Pi freezes with nothing in `kmsg` (the Spreadsheet's first 1060-pixel window did exactly that). wtk's Root checks it: the app stops with wtk: `the window could not be made in kmsg`.
- **Kernel widgets:** `kapi_add_button/label/checkbox/textbox/progress/slider/textarea/scrollbar_v/scrollbar_h/icon(...)` return an unsigned long handle. Manipulate them with `kapi_widget_set_text/get_text, get_checked, get/set_value, set_rect` (move/hide by setting `w=h=0`), `set_icon`.
- **Event loop:** either `kapi_wait_for_exit()` (blocking, simple), or your own loop `while (!kapi_should_exit()) { ...; kapi_pump_events(); kapi_present(); kapi_msleep(16); }` when you animate the canvas yourself.
- **App-drawn UI:** to draw text over your canvas, use `kapi_draw_text(x, y, s, color) + kapi_font_width/height()`. Capture the keyboard with `kapi_set_key_handler(fn)` (`GUI_EVENT_KEY` events, `KEY_*/ASCII` values) and the "outside-widget" clicks with `kapi_set_click_handler(fn)` (`GUI_EVENT_CANVAS_CLICK / ..._MOTION`, coordinates encoded in `value`). Cursor position relative to the window: `kapi_cursor_pos(&x, &y)`.
- **Yield anyway:** the scheduler preempts a busy app, so a loop that never yields no longer freezes the system, but it is treated as a CPU hog (short slices, served after the others) and its own window stops being redrawn and answering. Keep calling `present/msleep/pump_events/wait_for_exit` regularly.

See the demos `demoD.c` (widget gallery), `demoE.c` (textarea + scrollview), and the apps `tinypad.c`, `paint.c`, `mandelbrot.c` for complete examples.

A large app: Koton, the studio (`user/Apps/koton`)

Koton (the DAW: `docs/daw/README.md` has its plan and the user's decisions) is a **newlib** wtk app (`koton.elf` in `user/Makefile`: the engine, MeltySynth and the plugin host are separate objects in `Apps/koton/obj/`, the UI is headers included by `main.cpp`), with FreeType text (`ft/wtkface.h`: `ft_wtk_install ("DejaVu Sans", 13)` before the Root). Its parts:

Folder	What
<code>engine/</code> (namespace <code>kt</code>)	<code>kbase.h</code> (Vec, Str, .NET's seeded Random reproduced bit for bit, banker's rounding); <code>model.h/.cpp</code> (the song:

Folder	What
	tracks of items — a module after a silence, positions relative —, the chord track pinned last; <code>.sq</code> / <code>.kson</code> read and written with <code>json.hpp</code> ; <code>theory</code> (the modes, 35 qualities, degrees and colours, 30 cadences, voice leading, the next-chord suggestions, key changes); <code>gen_*.cpp</code> (every module rendered to notes: the chord styles and grids, the articulation, the drums, the euclidean rings, the melodic line engine); <code>compile</code> (the song flattened to events per track, the tempo map); <code>engine</code> (the audio engine: one MeltySynth per track, a lock-free command ring, the mix, the preview voice, the metronome — no allocation, no kapi call in <code>render</code>); <code>ai*</code> (Koton's AI: prompts, replies placed).
<code>synth/</code>	MeltySynth (C#) ported to C++: the SoundFont reader and the synthesizer, reverb and chorus.
<code>plug/</code>	The plugin host: plugins as processes (<code>kplug_proto.h</code> , <code>kplug.h</code>), rendered ahead through shared rings, effects with their latency compensated, generators, editors shown as applets.
<code>ui/</code> (namespace <code>kui</code>)	<code>palette.h</code> (the studio's colours as a wtk theme, the drawing helpers); <code>doc.h</code> (the song open, undo / redo as JSON snapshots, the timeline's edits); <code>audio.h</code> (the SoundFont found and loaded, the engine started on an app core , else a real-time thread, else the UI's tick; the song compiled when the document's revision changes); <code>arrange.h</code> (the arrangement: ONE widget the size of the view, drawing only what shows); <code>grid.h</code> (NoteGrid: every editor's grid — voice rows, drum lanes, the piano roll); <code>editor.h</code> + <code>ed_chord.h</code> , <code>ed_rhythm.h</code> , <code>ed_riff.h</code> (the block editors: a form of wtk controls in columns, a grid or a wheel on the right); <code>ops.h</code> (the editors' operations: the drum catalog, euclidean rhythms, the degree vocabulary, cadences, the next chord); <code>dialogs.h</code> ,

Folder	What
	<code>ai_dialog.h</code> , <code>chain.h</code> (a track's sound chain, a plugin's editor floating), <code>chrome.h</code> (the transport bar, the browser, the editor's host, the status bar).

Conventions worth keeping:

- **The document's revision:** every edit is `g_doc.checkpoint ()` (the undo step), the change, then `g_doc.changed ()`; the views and the audio host follow `g_doc.revision` (the song is compiled again once the edits pause — a drag recompiles a few times a second).
- **An editor never deletes itself:** a change that needs other controls sets `g_rebuildEditor`; the Root's tick makes the editor again from the model (the same after an undo: the pointers into the song are stale then).
- **The audio thread / core** makes no kapi call and allocates nothing; the UI posts commands (`Engine::post`) and frees the songs the engine hands back (`retired ()`).
- **Memory:** JSON documents are arenas (`json.hpp`), freed as a whole; the song and its modules are owned values (an `Item` owns its module, copies clone it). On the PC the app runs in the desktop simulator (`sh tools/tests/desktop_sim/shots.sh koton`) and under valgrind with the host allocator (`-DONYX_CPP_HPP -include new: onyxpp.hpp`'s allocator left out) — the engine's and the AI's own tests run under ASan / UBSan / LSan (`sh tools/tests/koton/engine_run.sh`, `ai_run.sh`, `synth_run.sh`). (ASan cannot run the simulator itself: its shadow memory takes the kapi table's fixed address.)
- **Static objects** that allocate must not be made before `main` (the kapi table is not there yet on Onyx): Koton makes its document in `main` (`g_doc` is a reference to storage constructed there).

Koton's plugins: processes over IPC (`kplug.h`, `kplug_proto.h`, `Apps/koton/plugin`)

Koton's instruments, effects and generators beyond the SoundFont are **plugins**, each a process of its own — the way the Control Panel hosts its applets: a crash in one never takes Koton down.

File	What
<code>user/kplug_proto.h</code>	The protocol (plain C, append-only like the kapi ABI: a message number or a field never moves; <code>KP_PROTO_VERSION</code> grows): the shared region (<code>KpShm</code>), the rings, the mailbox messages.
<code>user/kplug.h</code>	The plugin's side: a plugin is its parameters, a few callbacks and <code>KPLUG_MAIN (desc)</code> ; the runtime does the rest (the region, the handshake, the render thread, the parameters, the state, a generator's requests, the editor). Small DSP helpers (<code>KpAdsr</code> , <code>KpSvf</code> , <code>KpBiquad</code> , <code>KpNoise</code> , <code>kp_sin</code> , <code>kp_mtof</code> , <code>kp_db</code>).
<code>user/Apps/kp_<name>/</code>	A plugin: <code>main.cpp</code> + <code>plugin.json</code> (fm2,

File	What
	subsynth, pluck; delay, reverb, chorus, eq3, drive; arp, euclid, automaton — the catalogue: docs/04 , <i>Koton's plugins</i>).
<code>Apps/koton/plug/plughost.h</code>	The host: the catalogue, the processes, the engine's side of each, parameters, states, generators, editors, crashes.
<code>Apps/koton/plug/plugshm.h</code>	The engine's side of a plugin — <code>ShmSource</code> (a <code>kt::ExternalSource</code>), <code>ShmEffect</code> (a <code>kt::Effect</code>) — pure memory (they run on the app core).
<code>Apps/koton/plug/plugctx.h</code>	A generator's request (the context JSON) and reply (notes → a riff); a module's state (base64 of the plugin's JSON, as Koton's files).

Packaging. `SD:/koton/plugins/<name>/main` (the program, a newlib app with the FPU) and `plugin.json`:

```
{ "name", "kind": "instrument" | "effect" | "generator", "text", "vendor", "aliases": [other
ids: Koton's "koton.arpeggiator"...], "editor": { "w", "h" } (optional), "params": [{ "id",
"name", "min", "max", "default", "unit", "step" (1: whole numbers), "choices": [names] }] }
```

— the host lists the plugins from the manifests without starting them; a project stores a plugin by its folder's name (`PluginSlot::id`) or an alias. `user/Makefile` builds each `Apps/kp_<name>/main.cpp` into `Apps/kp_<name>/kp_<name>.elf` (`make plugins`; not a `user/*.elf`, which `make stage` would make an app of) and `make stage` copies it with its manifest to `sdcard/koton/plugins/<name>/`. The manifest must say what the program says: `sh tools/tests/koton/plug_run.sh --manifest <name>` prints the program's own list (the tests compare them).

Starting. The host makes the plugin's **shared region** — a surface (`kapi_surface_create`) of 512×224 "pixels" used as 448 KB of bytes, `KpShm` — fills its identity (kind, sample rate, lead, latency, the initial state as JSON), and runs `kapi_exec_as` ("`../main`", "`--kplug <surface> <host pid>`", "`kp.<name>.<n>`"). The plugin maps the region, loads the state, calls `prepare`, writes its parameter list into it, sets `ready` and sends `KP_HELLO`. The host is the IPC service `kotonplug` (one Koton hosts plugins at a time): a plugin whose host is gone ends by itself.

Real time without a round trip per block. The engine runs on an app core: no kernel call — it cannot wake anybody. But it is also the sequencer and knows the notes in advance, so it **renders ahead**:

- It counts every frame it outputs on a **stream clock** (never reset: a seek or a loop does not move it). For a track played by a plugin, a second cursor dispatches the song's notes **lead frames ahead** of the frame being played (4096 by default: 92.9 ms at 44.1 kHz), stamped on that clock, into the region's **event ring** (1024 events), and moves `want` (every event before it is in the ring) and `kick`.
- The plugin's **render thread** (`kapi_thread_priority`: real time) sleeps on `kick` (`kapi_wait_word`: the kernel re-reads sleeping words at each 10 ms tick, so the app core's write wakes it within 10 ms) and renders the frames [`done`, `want`] into the **output ring** (16384 frames), cutting its blocks at the events' frames (sample-accurate). The engine reads frame `P` of the ring when it plays `P`. The lead covers the worst wake (a tick) and the rendering many times.

- A **play** or a **seek** starts a new pre-roll: the plugins' voices are reset at the new ahead point, what they had rendered before is muted (a 3 ms fade), the played cursor waits `lead` frames, and both start together. A **loop** needs none (the ahead cursor wraps as the played one will). A **live note** (`CMD_EXT_NOTE_ON`, a MIDI keyboard on a plugin track) is stamped at the ahead point: heard `lead` late.
- An **effect**: the engine writes the track's dry block of frames `[t, t + n)` into the region's **input ring** (`want = t + n`), the plugin processes it into the output ring, and the engine takes back the frames `[t - latency, ...)` (4096 by default). While an IPC effect is connected, the engine **delays every other track by the same latency** (plugin delay compensation: a delay ring per latency, up to 4; two IPC effects in series on a track: twice as much) — the song stays aligned and is heard that much later; the playhead (`Engine::position`) is the song's frame heard now, the metronome clicks with it.
- A frame not rendered in time is **silence and a count** (`KpShm::underruns`, `Engine::pluginUnderruns`): the engine never waits. A plugin late behind the engine skips to where the engine reads (`lateFrames`).

The engine's side (`engine.h`): `ExternalSource` — `lead ()`, `noteOn / noteOff / allOff / reset (at, ...)` stamped on the stream clock, `flushTo (upTo)`, `render (l, r, n, at) → false` when not all was rendered in time; `Effect` — `latency ()` and `processAt (l, r, n, at)` besides `process` (a built-in effect keeps no latency); `CMD_EXT_NOTE_ON / OFF` (a live note on a plugin track); read by the UI: `renders` (the blocks rendered: a source or an effect taken out is free once it moved by 2), `streamClock`, `pluginUnderruns`, `extLead`, `pdCFrames`. With no plugin connected the engine renders exactly as before (the same samples: `tools/tests/koton/engine_run.sh`).

The messages (mailboxes; the requests' payloads and replies in the region's `data[]`, 128 KB of JSON, a request posted with `kp_req_post` and answered when `repSeq` = its number — the mailbox message is only a doorbell):

Message	Direction	What
<code>KP_HELLO</code> (100)	plugin → host	<code>KpHello {version, kind, nparams, shm}</code> : it is up.
<code>KP_SET_PARAM</code> (101)	host → plugin	<code>KiParamMsg {index, value}</code> .
<code>KP_PARAM_CHANGED</code> (102)	plugin → host	Its editor moved a parameter.
<code>KP_STATE_GET / KP_STATE_SET</code> (103 / 104)	host → plugin	Its state, JSON: <code>{"v":1, "params": {id: value...}, ...its own keys}</code> (a flat <code>{id: value}</code> , Koton's, is read too).
<code>KP_GENERATE</code> (105)	host → plugin	A generator: the context → <code>{"notes": [[start, len, MIDI note, velocity]...]}</code> .
<code>KP_PARAMS</code> (106)	host → plugin	Its parameter list (the <code>plugin.json</code> shape).
<code>KP_EDITOR</code> (107)	host → plugin	<code>KpEditor {surface, w, h, themed, window,</code>

Message	Direction	What
		button, field, accent): draw your editor into that surface (the host's colours), as an applet — then the applet protocol unchanged (applet_proto.h: AP_HELLO, AP_PRESENT, AP_EXIT back; AP_PTR, AP_KEY, AP_CLOSE in).
KP_DIRTY (108)	plugin → host	Its state changed otherwise than by a parameter.
KP_BYE (109)	both	Please end / it is ending.

(The plugin's editor is drawn by the plugin's runtime itself into the surface — wtk widgets under a panel that adopts the surface — rather than by wtk's applet mode, whose `Root` checks the Control Panel's service.)

Writing a plugin (user/Apps/kp_<name>/main.cpp):

```
#include "kplug.h"
enum { P_CUT, P_RES, NP };
static const KiParamDef P[NP] = {
    { "cutoff", "Cutoff", 20, 18000, 1200, "Hz", 0, 0 }, // id, name, min, max, default,
    { "res", "Resonance", 0.5f, 12, 1, "", 0, 0 },
};
static KpSvf s_f[2]; static int s_rate;
static void prepare (int rate) { s_rate = rate; }
static void setParam (int, float) { for (int c = 0; c < 2; c++) s_f[c].set (kp_param (P_CUT),
kp_param (P_RES), s_rate); }
static void process (float *l, float *r, int n) // an effect: in place
{ for (int k = 0; k < n; k++) { l[k] = s_f[0].tick (l[k], 0); r[k] = s_f[1].tick (r[k], 0); } }
static const KpDesc desc = { .name = "Filter", .kind = KP_EFFECT, .params = P, .nparams = NP,
                             .prepare = prepare, .process = process, .setParam = setParam };
KPLUG_MAIN (desc)
```

- An **instrument** gives `noteOn` (note, velocity), `noteOff`, `allOff` (hard) (hard: a seek, cut every voice) and `render` (l, r, n) (it writes n frames); an **effect** `process` (l, r, n); a **generator** `generate` (const KpContext &, const float *params, KpNotes &out) — the context: the block's length and absolute position, the key (tonic, mode, its scale), the meter (`ternary`), the tempo, the chords under the block (each its root, Koton's quality, its intervals `iv`, Koton's plugin contract `basic` / `biv`, its bass: `chordAt` (beat)), a seed and the module's whole state; the parameters of *that module* are in `params` (the process serves every block using the plugin); `out.add` (start, length, MIDI note, velocity) in beats from the block's start.
- **Threads:** `prepare`, `setParam`, the notes and `render` / `process` run on the **render thread** — no allocation, no lock, no wait, no kernel call (a NaN or a blow-up is zeroed before the mix anyway);

kp_param (i) reads a parameter there (changes arrive between blocks, setParam tells). generate, saveState / loadState (its own keys beyond params) and the editor run on the main thread.

- **The editor:** made from the parameters when the plugin gives none — a knob per parameter, a drop-down for choices, a check box for {"Off", "On"} —, laid out in the size the host asks (PlugHost::editorSize guesses it; editorW / editorH or the manifest's editor say better). An own editor (desc.editor (root, w, h)) builds wtk widgets under root and binds parameters with kp_knob / kp_choice / kp_toggle (they follow a change from the host); kp_set_param (i, v) moves one (the host is told), kp_dirty () says the state changed otherwise.
- **Tests on the PC:** define KPLUG_DSP_ONLY and KPLUG_TEST_SYM=<name>; no kernel, no wtk; KPLUG_MAIN exports a KpTestApi (attach to a region, one render pass, a request, a parameter) — see tools/tests/koton/plugin_test.cpp.

The host, for an app (the UI thread — the one that posts the engine's commands; the generator hook may run on another):

```
PlugHost host;
bool PlugHost::init (int sampleRate);           // false: no plugins here (the service
taken, an old kernel, the simulator)
int PlugHost::scan (const char *dir = KP_DIR);   // the catalogue: count (), info (i),
find (id or alias), countKind (kind)
void PlugHost::attach (Engine *e);               // (0: the engine was deleted)
void PlugHost::installGeneratorHook ();          // kt::g_generatorHook -> generate ():
one process per generator plugin,                // its replies cached by request;
ended when unused for a minute
bool PlugHost::syncTrack (int track, const Track &t); // its instrumentPlugin and inserts
(slots 0..3) made, kept, replaced,                // connected (a disabled one: kept,
disconnected); a running one keeps                // its live state (after an undo:
setState)
void PlugHost::releaseTracks (int from = 0);      // the tracks' instances from `from` on,
ended
void PlugHost::poll ();                          // every UI tick: the mailbox
(handleMessage (from, type, data, len) for an      // app that reads it itself -- the
rest to `foreign`) and tick ()
PlugEditorView *PlugHost::openEditor (PlugInstance *p, wtk::Widget &parent, int x, int y, int
w, int h);
PlugEditorView *PlugHost::openGeneratorEditor (const GeneratorModule &m, wtk::Widget &parent,
int x, int y, int w, int h);
bool PlugHost::pullGeneratorState (PlugInstance *p, Project &song); // at its onParam /
onDirty: the module's state from it
void PlugHost::closeEditor (PlugEditorView *v);   // (deleting its parent does too)
void PlugHost::saveTrack (int track, Track &t);   // before saving the song: the live
states into its PluginSlots
void PlugHost::shutdown ();                       // the engine stopped (or deleted:
attach (0) first)
```

and, lower down: create (id, stateJson, wait), destroy, restart (a crashed one, its last state, reconnected), connectInstrument (p, track), connectEffect (p, track, slot), disconnect, setParam, getState, setState, generator (id), setLead / setLatency (1024 .. 8192 frames, for the instances made after); a PlugInstance says its info () (the manifest), state () (PLUG_STARTING / READY / FAILED / CRASHED / ENDING), error (), param (i) (its value now), underruns (), dspUs (), track () / slot (). The callbacks onCrash (a plugin died or hung 3 s: an effect is taken out of its track, which goes on dry; an

instrument's track is silent), `onParam` (its editor moved a parameter), `onDirty`. A plugin process is ended (`KP_BYE`, killed a second later) when its slot is emptied, when a generator is unused for a minute, and at shutdown.

Limits: the kernel's 64 surfaces for the whole system (a plugin takes one, two with its editor open); 128 instances a host; 64 parameters a plugin; 1024 events in flight; 128 KB of JSON a request; the delay compensation up to 16128 frames (366 ms: three IPC effects in series on a track at the default latency) and four different latencies at once; a plugin's fault halts the Pi like any app's (§12). The editors' *Listen* (the engine's preview voice) plays a plugin track's block with the SoundFont.

Tests: `sh tools/tests/koton/plugin_run.sh` — under ASan / UBSan / LSan: the rings with a producer and a consumer thread; the engine rendering ahead with fake sources (the pre-roll, a note heard exactly when played, a loop, a seek while playing, a live note, an underrun, the delay compensation and the position heard); every plugin's DSP to `/tmp/koton_plugin_<name>.wav` (no NaN, no silence, the parameters' extremes); the generators from a project's context (the arpeggiator's notes are the chords' tones; Koton's states; twice the same); the states' round trips and the manifests; a real-time run (the engine paced like the audio pump, `kp_fm2` and `kp_delay` on threads that sleep between 10 ms ticks: no underrun). Then `plugin_host_run.sh`: the host itself and the plugins' kernel side (`kplugin_main`, the render thread, the editor) over the desktop simulator's stand-in kernel with a small multi-process layer (processes are threads): `syncTrack`, a song played through them, a parameter, the states, the editor drawn and a knob dragged, a generator block and its cache, a plugin killed and started again, the shutdown (`KPLUG_SHOT=prefix` writes the editors' pictures).

7. Writing a `/bin` tool

A `/bin` tool follows the **same EL1 app model** but reads `stdin`, writes `stdout`, and exits (no window). It is composable via the terminal's pipes.

```
#include "kapi.h"
#include "applib.h"      /* ax_puts, ax_putln, ax_strlen, ax_itoa, ... */

int main (void)
{
    char args[128];
    kapi_get_args (args, sizeof args);    /* la ligne d'arguments (chaîne) */

    /* Lire stdin et Le réécrire en majuscules, par exemple : */
    char buf[256];
    int n;
    while ((n = kapi_stdin_read (buf, sizeof buf)) > 0)
    {
        for (int i = 0; i < n; i++)
            if (buf[i] >= 'a' && buf[i] <= 'z') buf[i] -= 32;
        kapi_stdout_write (buf, (unsigned) n);
    }
    return 0;    /* code de sortie récupérable via kapi_wait dans l'appelant */
}
```

- `kapi_get_args(buf, size)`: the entire argument line as a **single string** separated by spaces (there is no `argv` array; parse the first word yourself, etc.).
- `kapi_stdin_read(buf, n)`: reads the task's stdin (0 = EOF). `kapi_stdout_write`: writes stdout.
- To read a file passed as an argument: `kapi_open/read/close`. To list a directory: `kapi_opendir/readdir/closedir`.

The existing tools to study: `ls`, `cat`, `grep`, `wc`, `echo`, `page`, `rm`, `mkdir`, `touch`, `cp`, `mv`, `ps`, `kill`, `run`, `keyb` (in `user/bin/`).

`memset` / `memcpy` in freestanding apps

Apps built with `-ffreestanding -nostdlib` (every wtk app and the plain `/bin` tools) have no `libc`, yet GCC may still emit calls to `memset`/`memcpy`/`memmove` on its own (e.g. `char buf[64] = ""`; struct copies). Since ABI v36 the kernel exposes Circle's implementations in the `kapi` table; `user/kapi.h` defines them as weak `kapi_memset`/`kapi_memcpy`/`kapi_memmove`, and `user/Makefile` / `user/bin/Makefile` alias the C names onto them at link time (`KAPI_ALIASES: -Wl,--defsym,memset=kapi_memset ...`). A new freestanding link rule must add `$(KAPI_ALIASES)`; newlib programs must not (they keep newlib's versions).

The AI helper `/bin/llm` and Koton's AI composition (`engine/ai.h`)

Koton's "compose with AI" is split like Lisa + `/bin/groq`: the app builds the prompts and places the reply; a newlib + mbedTLS helper, `/bin/llm` (`user/bin/llm.cpp`, in `TLS_PROGS`), does the HTTPS.

- `user/Apps/koton/engine/ai.h` (namespace `kt`; `ai.cpp` = the prompts, `ai_place.cpp` = the replies) is a port of Koton Studio's Engine/AI (`AiArrangement*`, `AiArrangementPlacer`, `AiPolyArrangement`, `AiPolyrhythm`, the clients) and the `ChordModelOps` / `TimelineHelper` parts they call (`AddAiChord`, `ApplyAiDrum`, `ApplyAiRiff`, `ChordsUnder`, `AddSectionMarkers`). The **French prompts are Koton's, byte for byte** (they are tuned; `tools/tests/koton/ai_run.sh` looks every generated line up in Koton's C# sources when they are on the machine).
 - `AIRequest` — kind (`AI_COMPOSE` a new piece, `AI_DEVELOP` develop a theme riff after the end, `AI_ADD_TRACK` one voice over the whole piece, `AI_ADD_DRUMS`, `AI_DRUM_GROOVE` one drum module, `AI_RIFF` one riff (+ a progression when no chord is under it), `AI_POLYRHYTHM` a polyrhythmic piece), `style`, `intention`, `measures`, `fullMelody` (riffs vs rhythm-only melodic lines), `drums`, `chordsVoice`, `polyChords`, `polyDrums`, `english` (the labels the model writes), `track` / `item` (the theme, the drum module, the riff; -1 = a new one / the last riff).
 - `aiBuildPrompt` (`project`, `req`, `system`, `user`) — including the whole piece as text for "add a track" (every track's notes in beats, the chords as [`bar`, `degree`, `quality`]) and the theme for "develop"; `aiFullPrompt` = "Copy the prompt" (works with no key: paste the answer back).
 - `aiCheckReply` (`req`, `text`, `summary`, `cap`) — parse only, a one-line summary for the dialog.
 - `aiApplyReply` (`project`, `req`, `text`, `err`, `cap`) — tolerant parsing (a `` fence, text around the object, trailing commas, notes as [`a`, `b`, `c`] or objects, numbers as strings, one object for a list); placement as Koton's placer: the chord track pinned last with one degree-locked chord a bar (colour from `colourForQuality`; a chord that is not its degree's own — V in minor, V/V — stays fixed), one chord articulation a section on an "Accompaniment" track (the AI's one-bar motif, saved as a user chord style, and its melodic cell), melodic lines or riffs (`PlayRiff` + `Riff`, the out-of-harmony notes snapped) grouped by role in 4-bar blocks, drum modules from the motif (`CompressPeriodic`), polychords / polydrums in 1–4-bar modules, the sections as markers, the key / meter / tempo. `AI_COMPOSE` / `AI_POLYRHYTHM` replace the project; on an error the project is unchanged. Deliberate differences with Koton are marked **Onyx**: in the sources (the default track names follow `english`; "Do" / "Ré" and `7#9` are read right; a develop's accompaniment lies under the development).

- `aiBuildRequestJson` (provider, model, key, system, user, temperature, thinking, writer[, url]) — the request for `/bin/llm`; `aiParseLlmOutput` (out, len, text, error, &lastProgress) — its answer; `aiBuildFetchJson` (url, outPath, writer) / `aiParseFetchOutput` (out, len, &bytes, error, &progress) — the same for its download mode.
 - `g_aiProviders`, `aiProviderLabel`, `aiProviderDefaultModel` — Koton's providers.
- `/bin/llm [request.json] [-o result.json]` reads ONE JSON document (stdin, or the file): `{ "provider": "gemini"|"groq"|"mistral"|"claude"|"openai-compatible" (+ "deepseek", "grok", "openai"), "model", "key", "system", "user", "json": true, "temperature": 0.7, "thinking": -1, "url"? , "maxTokens"? , "timeout"? }` and writes ONE JSON line: `{"ok":true,"text":"..."}` or `{"ok":false,"error":"..."}` (exit code 0 / 1). Gemini: `POST .../v1beta/models/{model}:generateContent, x-goog-api-key, responseMimeType: application/json, thinkingConfig.thinkingBudget` when `thinking ≥ 0`; the text = `candidates[0].content.parts[*].text`, `MAX_TOKENS` = an error. Groq, Mistral, DeepSeek, Grok, OpenAI and any OpenAI-compatible url: `chat/completions` with `response_format: json_object`. Claude: the messages API (`max_tokens 32000`, no temperature). A **download** mode: `{ "fetch": "https://...", "out": "SD:/..." }` → the body written to `out` (redirects followed, folders made) and `{"ok":true,"bytes":N}` — Koton fetches its SoundFont with it. Progress lines go to `stderr` — on Onyx the same stream as `stdout` — as `llm: connecting to ..., llm: sending N bytes, llm: waiting for the answer (N s), llm: receiving N bytes [of M]`; the result is the last line starting with `{`. The response buffer grows (16 MB for an answer, 512 MB for a download; sized from `Content-Length` at once). The TLS certificate is **not verified** (no CA bundle on the card). After the handshake the TLS receive is made non-blocking (onyx_tls's own gives up after 20 s of silence; a thinking model may be silent for a minute). A busy model's answer (429, 500, 502, 503, 504 -- Gemini's free models say 503 "high demand" often, even for a short prompt) is asked again after 5, 10, 20 and 40 s (`llm: Gemini answered 503 (busy): asking again in 5 s (1/4)`) before its error is given.
- **From the app** (gtk, freestanding): build the prompt and the request, then


```
json::Writer rq (false);
kt::aiBuildRequestJson ("gemini", model, key, sys, usr, 0.7, -1, rq);
void *in = kapi_pipe (), *out = kapi_pipe ();
void *proc = kapi_spawn ("SD:/bin/llm", "", in, out);
// write rq.data () to `in` in one go (a pipe holds 8 KB, the write yields until llm
has read it:
// llm reads all of its stdin first, as groq does for Lisa), then kapi_stream_eof (in)
// each tick: kapi_stream_read_nb (out, ...) into a growing buffer (the answer can be
100+ KB);
// kt::aiParseLlmOutput (buf, n, text, error, &progress) -> the status bar shows
`progress`
// when kapi_proc_done (proc): kapi_wait, close the pipes, then
// if (kt::aiParseLlmOutput (buf, n, text, error)) kt::aiApplyReply (project, req,
text, err, sizeof err);
```

In Koton (`ui/ai_dialog.h`) this runs inside the open dialog (`runLlmIn`: a nested pump, the dialog redrawn, Cancel sets the flag that kills `llm`); `aiCheckReply` sums the reply up there, and *Apply as a new song* places it on a fresh `Project` (`COMPOSE` / `POLYRHYTHM`) or a copy of the song (the additions), taken by `Doc::adopt` -- untitled, dirty, no undo history -- after `askSave`. The app keeps the provider, the model, the API key and the dialog's last request (`aiLast`: kind, style, intention, bars, the options) in `SD:/koton/settings.json` (plain text on the card — the key field's tooltip says so); its dialog is `ui/ai_dialog.h`.

- **Tests on the PC:** `sh tools/tests/koton/ai_run.sh` — every kind of prompt, canned replies shaped like Gemini's (placed, then checked: tracks, chords through `chordAt` and a key change, every module rendered, `compileSong`, a `.kson` round trip), malformed replies, the `/bin/llm` protocol per provider (`#define LLM_PROTO_ONLY + include llm.cpp`) and a 60+ KB reply end to end, under ASan / UBSan / LSan; then the AArch64 compile and `make -C user/bin llm.elf`.

8. The `applib.h` library

`user/applib.h` is **header-only** (no libc). It provides:

- **Strings:** `ax_strlen`, `ax_streq`, `ax_strcat(dst, cap, &pos, src)` (concat without overflow), `ax_app_path(dst, cap, name, suffix)` (builds `SD:apps/<name><suffix>`), `ax_itoa(v, buf)`, `ax_fmt2(d, v)` (2-digit decimal).
- **Console:** `ax_puts(s)`, `ax_putln(s)` (to stdout).
- A minimal **.ini reader:** `app_ini_load("config.ini")` (from the app's folder via `kapi_app_dir`) or `app_ini_load_path("SD:skins/theme.txt")`; then `app_ini_get(section, key, default)` and `app_ini_get_int(...)`. Sections `[xxx]`, lines `key=value`, comments `;/#`.
- **App-drawn widgets** (not kernel widgets): `ax_dropdown` (drop-down list) and `ax_colorpick` (palette-based color picker), with `*_draw(...)` and `*_click(...)`. The app draws them in its canvas and routes the clicks via its `set_click_handler`. Used by `theme.c` and `mandelbrot.c`.

There is also `user/httpc.h` — a header-only **HTTP/1.0 client** over the v21 TCP socket calls. It does **no allocation** (the caller passes the response buffer, so all memory stays in the app's address space): `http_get(url, buf, cap, &resp)`, `http_post(...)`, or the general `http_request(method, url, headers, body, len, buf, cap, &resp)`. Plain HTTP only (no TLS). Used by `/bin/wget`. This is the recommended pattern for application-level protocols: build them in a user library on top of the kernel's transport `kapis`, rather than adding them to the ABI.

For **REST / web-API** clients there is a reusable C++ class, `user/http.hpp` (`HttpClient/HttpResponse`) — a header-only, freestanding **HTTP/1.1** client that works in any app (integer-only or newlib). It adds, over `httpc.h`: custom default headers (chainable `.bearer(token)`, `.accept(type)`, `.header(name,value)`), JSON helpers (`post_json/put_json`), response-header lookup (`r.header("Content-Type", out, cap)`), and **chunked** transfer decoding. Same no-allocation model (the caller passes `char buf[N]`; the body points into it, NUL-terminated). Errors are a negative `r.status` (`HttpError`). Example: `HttpClient api; api.bearer(tok).accept("application/json"); auto r = api.get(url, buf, sizeof buf); if (r.ok()) ...`. See `/bin/httpget` for a working demo. **HTTPS:** the class has a transport seam (`http://` = plain `kapi` TCP; `https://` = TLS). TLS is provided by `user/tls/onyx_tls.hpp` — **mbedtls ≥ 3.6.3** over the `kapi` sockets, with a **buffered** BIO (Circle's `CSocket::Receive` discards the remainder of a TCP segment on a short read, so we read whole segments) and **software-only crypto** (the Pi 4's Cortex-A72 has no ARMv8 crypto extensions). **Verified end-to-end on real hardware** — `httpsget` downloads real pages. Same model NetSurf uses (HTTPS from an external stack, libcurl+OpenSSL). To enable it in a **newlib** app: `#define ONYX_HTTP_TLS` before `#include "http.hpp"` and link the cross-built mbedtls libs — `make -C user/tls` then `make -C user/bin MBEDTLS_DIR=../tls/mbedtls` (see `user/tls/README.md`). The freestanding default (no `ONYX_HTTP_TLS`) keeps `http://` only and returns `HTTP_ERR_HTTPS` for `https://`. Demo: `/bin/httpsget`. **Not yet secure:** the RNG is a software PRNG (not the HW RNG, which stalls on the Pi 4) and certificate verification is OFF — see the TLS README.

A full HTTP client app: Courier (`user/Apps/courier`, the user guide's *Courier, the HTTP client*) — Postman for Onyx. It keeps `http.hpp`'s Transport (plain / TLS) but has its own engine (`net.h`): the request prepared from the model (`{{variables}}`) resolved, auth, raw / urlencoded / multipart / binary bodies,

the cookie jar), sent on a **thread** (`kapi_thread_create`; the result handed to the UI with `kapi_post`; `Job::cancel` polled by the read loop), the response read into a growing buffer (to 16 MB; the end found by Content-Length or the last chunk, not only by the close), gzip / deflate inflated with `img_inflate`, redirects followed with their cookies, timed with `kapi_clock_us`. Its data are Postman's own formats (`model.h`: collection v2.1, environment), read and written with `json.hpp`. The pieces: `util.h` (Str, Vec), `model.h`, `vars.h`, `net.h`, `tools.h` (tests, snippets, cURL, JSON / XML formatting), `widgets.h` (LineEdit with `{{variable}}` pills, CodeEdit — a code editor with colours, undo, selection —, KVTable with Bulk Edit, TabBar, DocTabs, Btn, Choice), `views.h`, `rail.h` (the sidebar), `dialogs.h`, `main.cpp`. Built by `user/Makefile`'s `courier.elf` rule: a newlib app as Writer (FreeType's text through wtk) linking mbedTLS (`COURIER_MBEDTLS`, default `third_party/mbedtls-3.6.3`). On the PC it builds with `-DCOURIER_NO_TLS` (`shots.sh courier`).

For **images** there is a reusable decoder, `user/img/image.hpp` (`onximg::decode(data, len, &w, &h)`) — built on the cross-compiled **zlib + libpng + libjpeg**. It sniffs the format (PNG signature / JPEG SOI) and decodes a byte buffer into a freshly `malloc`'d array of `0xAARRGGBB` pixels (8-bit alpha in the top byte, which the canvas ignores when blitting). Same split as TLS: the libraries are cross-built once (`make -C user/img`, sources pinned in `user/img/README.md` — zlib 1.3.1, libpng 1.6.44, libjpeg IJG v9f), the Onyx glue is header-only. It is a **newlib** component (uses `malloc` + the libs), so it is OPT-IN: `make -C user/bin IMG_DIR=./img` builds the `/bin/imgtest` demo (decodes an embedded PNG and prints its size). This is the same model NetSurf uses — link `libpng/libjpeg/zlib`, decode behind one wrapper. Note: `image.hpp` is for full-colour web images; keep `user/bmp.hpp` for the magenta-keyed `0x00RRGGBB` icons loaded from SD.

For a **NetSurf-style framebuffer GUI** there is an Onyx **libnsfb** surface backend, `user/nsfb/onxy_surface.c`. `libnsfb` is NetSurf's framebuffer abstraction (its software plotters draw into a surface buffer); this backend makes an Onyx window **content canvas** that surface: `initialise` → `kapi_create_window` (the `0x00RRGGBB` canvas is the framebuffer), `update` → `kapi_present`, and input bridges Onyx's callback-driven pointer/key events (`kapi_set_pointer/key_handler` + `pump_events`) into `libnsfb`'s poll-style event queue. The format is `NSFB_FMT_XRGB8888` — on little-endian the plotter packs `0x00RRGGBB`, exactly the canvas layout (no R/B swap). The vendored `libnsfb` is **unpatched**: the backend registers under the name "onyx" (resolved with `nsfb_type_from_name("onyx")`) via a constructor that Onyx's `crt0` runs from `.init_array`. Cross-built once (`make -C user/nsfb`, pinned in `user/nsfb/README.md`), then OPT-IN: `make -C user/bin NSFB_DIR=./nsfb` builds the `/bin/nsfbdemo` demo (draws shapes through `libnsfb` and tracks the cursor). `libnsfb.a` is linked `--whole-archive` so the surface's registration constructor is not dropped.

The **NetSurf core library stack** also cross-builds for Onyx — `user/netsurf/` builds `libwapcaplet`, `libparserutils`, `libnsutils`, `libnsgif`, `libnsbmp`, `libhubbub` (HTML), `libcss` (CSS) and `libdom` (DOM) against newlib (`make -C user/netsurf`, versions pinned in `user/netsurf/README.md`). The whole stack **links clean** — no undefined symbols — against `crt0libc` + `onxy_syscalls` + newlib. Three Onyx-side fixes made it self-contained: `libparserutils` is built `-DWITHOUT_ICONV_FILTER` (use its own charset codecs; newlib has no `iconv`), everything is built `-fcommon` (the code predates GCC 10's `-fno-common`), and `pread/pwrite` were added to `onxy_syscalls.c`. Code that the upstream buildsystem normally generates with host tools is reproduced in the Makefile: perl for the `libparserutils` charset aliases and `libhubbub` entities, a host-compiled `gen_parser` for the `libcss` property parsers, and a gperf-free element-type table for `libhubbub` (`user/netsurf/gen/`). This is NetSurf brick 7. Brick 8 — an Onyx **fetch scheme handler** (`user/netsurf/onxy_fetch.c`) — drives the NetSurf fetch API over the Onyx TCP kapis (+ gzip via zlib), the curl-fetcher's role without `libcurl`. Brick 9 wires the whole **NetSurf core + its framebuffer frontend** to the `user/nsfb` "onyx" `libnsfb` surface + the `user/img` decoders: `make -f user/netsurf/netsurf-app.mk` stage builds `netsurf.elf` (182 TUs + the libs) and installs it as the NetSurf desktop app. **It runs on Onyx**

— the window opens and real web pages render through the full HTML/CSS engine (currently slow; the `onyx_main.c` entry shim passes `-f onyx` to select the window surface). A console `nstest` (`netsurf-app.mk nstest`) smoke- tests each library brick. See `user/netsurf/README.md`. NetSurf has since been changed a great deal for Onyx — its fonts (FreeType, web fonts, metric-compatible stand-ins), CSS3 in `libcscs`, flexbox / grid / baseline layout, anti-aliased CSS3 painting, the native window: `06-NETSURF-CHANGES.md` lists the changes. A change is checked on the PC first: `sh tools/tests/netsurf/shot.sh <url|file> <out.png> [WxH]` renders a page with NetSurf built for the PC (the desktop simulator), and `sh tools/tests/netsurf/chrome.sh <url|file> <out.png> [w] [h]` the same page in Chromium.

And `user/uikit.h` — a **retained-mode widget toolkit** drawn entirely in the app's canvas, driven by the kernel's **pointer stream** (ABI v22: `set_pointer_handler` → `GUI_EVENT_PTR_MOVE/DOWN/UP/ENTER/LEAVE/WHEEL` with client coords; `GUI_EVENT_PTR_WHEEL` carries a signed notch delta in the `lvalue` wheel field, decoded with `GUI_PTR_WHEEL` — scrollbars/text areas in both toolkits scroll on it). Same memory model as the rest: widgets live in a caller-provided **fixed pool** (`ui_widget pool[N]` in the app's `.bss`, freed automatically on exit — no user `malloc`, no kernel object behind a widget). `ui_init`, `ui_button/ui_label/ui_checkbox/ui_textbox`, `ui_on_event` (fed from the app's pointer + key handlers), `ui_draw`. The **textbox** is a single-line editor: caret, Backspace/Delete, arrows, Home/End, `Tab` to move focus, an optional password mask (`ui_set_password`), read with `ui_get_text`. `tinycalc` (buttons) and `wpaconf` (a form of textboxes) use the toolkit. This is the forward path for widgets: new ones are added here, in userland, with no kernel/ABI change. The older **kernel-drawn widgets** (`add_button...`, §earlier) still work and coexist; apps choose one model per window.

Dynamic memory + C++ apps

Apps have **no `malloc` by default** — they use static buffers + the stack (both in the app's address space). For dynamic memory, include `user/umm.h`: a small user allocator (size-class free lists + a `kapi_sbrk` arena). `umm_malloc` / `umm_free` / `umm_calloc` / `umm_realloc`. The heap lives at `USER_HEAP_BASE` (10 GB); its pages are owned by the address space, so they are **freed automatically when the app exits** and show up in the app's page count (`ps / memmon`). `kapi_sbrk` is the underlying primitive (rarely called directly). `/bin/heaptest` exercises it.

C++ apps are supported (freestanding subset — no exceptions, no RTTI, no STL):

- Name the source `*.cpp`; the user Makefile builds it with `g++ (-fno-exceptions -fno-rtti -fno-threadsafe-statics -fno-use-cxa-atexit)`.
- Include `user/onyxpp.hpp`: it defines `operator new/delete` (on `umm`) and the runtime stubs (`__cxa_pure_virtual`, `__dso_handle`, `__cxa_atexit/atexit` no-ops). Global constructors run via `crt0.S` (the `.init_array` walk); static destructors are **not** run (the app exits and its address space is reclaimed).
- Classes, inheritance and virtual methods (vtables) work; `new/delete` go through the user heap. No `std::string/std::vector` — write small containers on `umm` as needed. See `user/cppdemo.cpp` for a working example.

9. Packaging an app: `.app`, icons, `config.ini`

Layout of an application on the card (produced by `make stage`):

```
SD:apps/<nom>.app/  
  main          l'ELF de l'app, sans extension (obligatoire)  
  icon.bmp      icône 40×40, BMP 24 bpp ; le magenta 0xFF00FF est transparent (optionnel)
```

app.txt	métadonnées (display name + catégorie) lues par le launcher (optionnel)
config.ini	configuration de l'app, lue via app_ini_load() (optionnel)

An app may also be written in **BASIC**: `main.bas` (or a compiled `main.bax`) instead of `main`. The kernel only loads ELF's: `user/launch.h` resolves the rest from `SD:/etc/runners.ini` ("extension = program", e.g. `bax` = `SD:/bin/basic`, `gb` / `gbc` = `SD:/apps/gbemu.app/main`, `gba` = `SD:/apps/gbaemu.app/main`, `nes` = `SD:/apps/nesemu.app/main`, `sfc` / `smc` = `SD:/apps/snesemu.app/main`, `wad` = `SD:/apps/doom.app/main`): `lx_launch` (name, args) starts an app (its `main`, else the first `main.<ext>` with a runner), `lx_open` (path, args) a program file (an ELF, or by its runner), both through `kapi_exec_as` so the process is named after the app. The launchers use it: the menu bar, `run`, `fileassoc.h` (File Viewer, the dock), the dock, the Game Library. A new format = one line in `runners.ini`. An app written in BASIC and shipped compiled is listed in `BASIC_APPS` of `kernel/Makefile`: `make stage` compiles it with `tools/basc` (the host build of the same compiler) to `apps/<name>.app/main.bax` (Arkanoid). See *Onyx BASIC* below.

The **app name** is the base name of the `.app` folder (without the suffix). That is what you put in `/etc/autostart` / `/etc/quicklaunch.txt` and what `kapi_list_apps` returns.

app.txt — friendly metadata for launchers (key = value, no section, read with `app_ini_load_path` / `app_ini_get(0, ...)`). Every app under `SD:apps/` ships one:

```
name      = Text Editor           ; display name shown under the icon
category  = Productivity          ; Productivity, Internet, Graphics, Games, Demos, Settings,
Emulators, Shell
stack     = 8M                    ; optional: the app's stack (bytes, K or M), read by the
KERNEL
```

stack is the one key the kernel reads (`AppStackSize`, `kernel/kernel.cpp`, when it creates the app's task): an app runs on its kernel task's stack, **256 KB** by default; a bigger one is asked for here — rounded up to 64 KB, at most 64 MB (smaller: ignored). The stack is kernel memory (it must stay mapped when the address space switches), taken from the kernel heap and reused after the app ends. NetSurf asks for 8 MB (its JavaScript engine may use 4).

The menu bar's **Onyx** menu and the dock's drawers group the apps by `category` and show their `name`. Three categories are **not listed** there: `Shell` (the desktop's own parts: `menubar`, `dock`, `notifyd`, `agenda`, `lock`, `ask`, `shell`... and the retired `panel`, `applist`, `shelf`), `Settings` (the Control Panel's applets: reached through it) and `Emulators` (reached through the Game Library, which starts the right one for a game). A shell component also creates its window with `WIN_FLAG_SYSTEM` (`kapi_create_window_ex` / the positioned `wtk::Root` constructor), so it is left out of `kapi_list_windows` (the menu bar's Open Windows, the dock's running dots) and shown on every workspace.

Icons — `tools/gen_assets.py` procedurally generates the 40×40 BMPs (BGR bottom-up, 4-byte padding) for all the apps (a document for `tinypad`, a calculator for `tinycalc`, a glider for `life`, etc.) and the "9 squares" glyph of the `apps` button in the panel. `tools/preview_icons.py` produces a PNG preview montage. Workflow: run `gen_assets.py` (writes the `icon.bmp` files into `sdcard/apps/<name>.app/`), then `preview_icons.py` to check visually.

config.ini — example read by `inidemo` / `voronoy` / `panel`:

```
; commentaire
greeting = bonjour
[app]
name = Mon App
version = 1
```

```
[display]
barwidth = 40
```

Screenshots and documentation

- **Screenshots (the real apps):** `tools/tests/desktop_sim/shots.sh` builds each documented app **for the PC** against the desktop simulator's stand-in kernel (`fakekapi.cpp`, wtk with its real image codecs and fonts), plays a short script of events (clicks, keys, menu commands; a canned shell session for the terminal `SIM_PIPE`, an IRC server `SIM_NET`, the mailbox messages a process would send `SIM_MBOX` — IRC's conversation window —, sample files from `tools/tests/desktop_sim/sd/SIM_OVERLAY`), dumps its window — the frame wtk drew and the client area — and writes `screenshots/<name>.png` (`shot.py`: the window alone, its rounded corners transparent) or lays several over the Voronoi wallpaper (`compose.py`: `desktop.png`, `menubar.png`, `volume.png`, `clock.png`, `wifimenu.png`, `dock.png`, `agenda.png`). Run `sh tools/tests/desktop_sim/shots.sh` (all) or `sh tools/tests/desktop_sim/shots.sh paint dock` (some); ~15 s, needs g++ and Pillow + numpy. **To add an app:** add it to `APPS` and a line `sim <app> <name> "<script>" ...; png <name>`. The script's steps: `wait`, `down / up / move / wheel X Y`, `rdown / rup`, `key CODE`, `mods N` (the modifiers held from then on: 1 Ctrl, 2 Shift, 4 Alt — Shift+arrows select...), `menu N` (the app's menu item N: its items counted from 0 in the order the app adds them), `winctl N`, `dump FILE`, `quit` (the window closed: the app's loop ends and what it does before leaving `main` runs — `exit` stops the process on the spot). Like the kernel, the simulator makes no window over 1024 × 768 (`kapi_create_window` returns 0). **Threads and the network:** the simulator runs an app's threads (`kapi` v67) as pthreads — `kapi_post`'s calls run at the main thread's next `pump_events`, a thread's `msleep` only sleeps (the script is the main thread's) — and with `SIM_REALNET=1` its TCP sockets are the PC's: an HTTP client against a local server (`python3 -m http.server...`), with `SIM_SLEEP=1` for the answers to come in real time. Without it, `SIM_NET` is what any connection receives (Courier's screenshots: a canned HTTP response). **The Pi's own binary on the PC:** `sh tools/tests/desktop_sim/elfrun.sh <app> [stack bytes]` (the same `SIM` script) runs `user/<app>.elf` — newlib and the code the Pi's compiler made — under `qemu-aarch64` with the simulator's `kapi` (`elfrun.cpp`: the ELF's segments at their addresses, the stack with a guard page); needs `g++-aarch64-linux-gnu` and `qemu-user`. Writer's and the Spreadsheet's are built with the apps' FreeType (the same sources, for the PC). (`nintendoemu.png` and `arkanoid.png` — an emulator, a BASIC program — still come from the older, simulated renderer `tools/screenshot/render.py`.)
- **Manuals** (a big app's user manual, for its users): `sdcard/manuals/<app>/<Name>.md`, its pictures in `images/` and its PDF `<Name>.pdf` next to it (the folder is `manuals`, not `Docs`: it would be the same folder as `docs/` on Windows' and FAT's case-insensitive disks). Written in English, in a small Markdown subset (headings, paragraphs, bold / italic / code, links, lists, quotes, tables, figures — a picture alone in its paragraph, an italic line after it its caption), the one `tools/manuals/build_manuals.py` documents at its top. Its pictures: a script per manual, `tools/manuals/ledger_shots.sh` (as `shots.sh`, the demo's data, the stand-in kernel's fixed day: the same pictures every time; Writer run on what Ledger printed), `annotate.py` adding numbered callouts in margins. The PDF: `python tools/manuals/build_manuals.py [its .md]` — Python only; a headless Chrome, Chromium or Edge prints it (`$CHROME` to choose one), A4, the docs' colours, Selawik (`sdcard/res/fonts`), bookmarks from the headings. Translations: `<Name>.<lang>.md` next to it (`Ledger.fr.md`, `Ledger.nl.md` → `Ledger.fr.pdf`, `Ledger.nl.pdf`), the same pictures (the apps' screens are in English: the manual names their buttons as on the screen, then translates them). Ledger's: `sdcard/manuals/ledger/Ledger.md`, `.fr.md`, `.nl.md`. Koton's: `sdcard/manuals/koton/Koton.md`, `.fr.md`, its pictures by `tools/manuals/koton_shots.sh` (the app

in the simulator on its demo song, 1920x1080, clicks at fixed places; the plugin editors are the pictures `tools/tests/koton/plug_host_run.sh` made, the simulator running no plugin).

- **Word/PDF exports:** `docs/build_docs.py` converts each `.md` in `docs/` into `.docx` (via `pandoc`) then into `.pdf` (via `Word/docx2pdf`), in `docs/exports/`. Run `python docs/build_docs.py` after any modification to the `.md` files. Prerequisites (once): `pip install python-docx docx2pdf py pandoc_binary`.
- **Reminder of the project rule** (see `CLAUDE.md`): any modification of a `kapi` function or of an app **updates the docs in the same go**, regenerates the affected screenshot if the visual changes, then regenerates the exports.

Onyx BASIC (the interpreter)

- **Core** (`user/basic/`): `bas.h` (API: `bas::compile`, `bas::run`, the `bas::Host` interface), `bascomp.cpp` (lexer + one-pass compiler to bytecode, with a pre-scan of the SUB / FUNCTION / DEF FN headers and of the TYPEs), `basvm.cpp` (the stack VM: tagged values, ref-counted strings, arrays and records -- a record shared at a store is copied first, so TYPEs have value semantics -- references (by-ref arguments, the address of an element or a field: `OP_REFG` / `OP_AADDRG` / `OP_FADDR`, stored through with `OP_STREF`), a frame per call), `basnum.cpp` (number formatting / parsing and the math functions, no libc). Numeric sub-types are a compile-time matter: values are doubles; a store into an INTEGER / LONG gets an `OP_CONV` (round half even, Overflow), a fixed string an `OP_FIXSTR`; a DOUBLE is only a printing precision (the compiler tracks `dblSeen` per expression). **Errors:** `VM::fail` fails the op; before the next one `trap ()` looks for the resume point (setjmp-like): with `ON ERROR GOTO` it unwinds the frames and the value stack and jumps to the handler; `RESUME` uses the statement table (`Program::stmts`) to restart or skip the statement that failed. **Events** (`ON TIMER` / `ON KEY`) are checked every 32 ops and after each blocking statement, and fire as a GOSUB (the interrupted expression stays on the stack). **CHAIN:** `bas::run` loops, compiling the next program and moving the COMMON values and the open files into the new VM. Portable C++ (only `new` / `delete`): built into `basic/libbasic.a` with FP/SIMD (`-fno-math-errno`), and on a PC for the tests.
- **Runtime** `/bin/basic` (`basic/runtime.cpp`): the Onyx `bas::Host` — console (stdio) or a `wtk::Root` window (a text/graphics framebuffer + wtk controls, pumped by the VM through `Host::poll`, `Root::attach ()`). A windowed app (`WINDOW`) asks the platform its colours (`ScreenHost::windowColours`: Onyx = the theme's `C_BG` / `C_TEXT`, the PC none) and takes them as its background, text and drawing colours unless the program set `COLOR` first. Options `-d <dir>`, `-i` (report errors to the editor: mailbox to the IPC service `qbasic`, payload `line\0message\0`).
- **Editor** `apps/qbasic` links `libbasic.a` for the syntax check.
- **Adding a function:** an entry in `BFNS[]` (`bascomp.cpp`: name, id, result type, argument spec `N/S/?`, `[]` = optional from here), a `B_*` id (`basint.h`), its case in `VM::builtin` (`basvm.cpp`); something the VM cannot do itself goes through a new `bas::Host` virtual (default no-op) implemented in `runtime.cpp`. Statements: `simpleStatement` (fixed arguments) or a dedicated `st*` parser, `S_*` id, `VM::statement`.
- **Keys:** the kernel delivers F1-F12 as `KEY_F1 .. KEY_F12` (0x110..0x11B); the runtime turns them into QBasic's `INKEY$` codes (`CHR$(0) + CHR$(59..68)`, 133, 134).
- **Runtime graphics** (`runtime.cpp`): a mode table (size, text cell height, pages, colours, display scale), up to 8 page buffers (draw `apage`, show `vpage`), a 256-entry palette (the VGA default; `PALETTE` recolours the pixels already drawn), a text-cell buffer (`SCREEN ()`), `VIEW PRINT` rows, a clip rectangle (`VIEW`), scanline `PAINT`, `readRect` / `writeRect` for GET / PUT; `FULLSCREEN` uses `kapi_fullscreen_begin` and scales the visible page (aspect kept; whole-number zoom when it

covers $\geq 85\%$). `PLAY "MB"` notes go to a 32-note queue that `bgTick ()` plays from `pump ()`; `KEYDOWN` maps its key to a `KEY_*` code for `kapi_key_held`. `PAD / STICK / STRIG` call `Host::padButtons / padAxis` (Onyx: `user/gamepad.h`; the PC: `winmm joyGetPosEx`).

- **3D** (`basic/bas3d.h`, `basic/bas3dscene.h`): the VM's `Scene3D` turns the 3D statements into vertices + batches in the layout of `kapi v53` (`G3Vertex = kapi_gpu_vertex3`, `G3Batch = kapi_gpu_batch`, same flag bits): the vertices stay in model space, each batch carries projection $\times$ view $\times$ model (a new batch when the matrix, the texture or the state changes); the shapes are lit on the CPU (the normal turned by the model matrix). `RENDER3D` calls `Host::render3d`; `ScreenHost` keeps a copy of each texture (`GRAB3D  $\rightarrow$  Host::texture3d`) and asks the platform (`gpuTexture / gpuRender`: Onyx's runtime = `kapi_gpu_texture / kapi_gpu_render` into the active page), else draws with `swRender` (the same semantics in software: near-plane clipping, perspective-correct, depth, culling, blending). Host test: `sh tools/tests/run_basic3d_test.sh` (a window-less `ScreenHost` saves each `RENDER3D` as a PPM and checks some pixels).
- **Compiled programs** (`basbax.cpp`): `saveBax ()` writes a `Program` table by table (little-endian; header "OBAX", the format and the VM's opcode / builtin / statement counts, so a `.bax` from another VM is refused), `loadBax ()` reads it back, `load ()` takes a file's bytes (a `.bax`, else source to compile) -- used by `/bin/basic (-c compiles)`, `CHAIN`, the PC `obcore.dll` (`ob_compile`, `ob_run` with a length). `runners.ini` sends `.bax` and `.bas` to `/bin/basic` (an app's `main.bax` before its `main.bas`). The tests run every program a second time through a `.bax` (`BAX=1`). **Add opcodes / builtins / statements at the end** of their enums: the counts in the header change, old `.bax` files are then refused cleanly.
- **Methods**: `SUB Type.Name` (the part before the last dot is a `TYPE`) is a procedure `TYPE.NAME` whose first parameter is `THIS`, the record by reference; `fieldPath ()` turns an unknown last part of a record path into `Ref.method`, and the call pushes the record's address (`emitAddr`) then the arguments (`callMethod`). `SUB Type.new` is the constructor: `DIM v AS Type (args)` and `NEW Type (args)` (`OP_NEWREC`: a fresh record) call it. Errors: `trap ()` keeps the `SUB` frames (each remembers its value-stack depth, `sp0`), so `RESUME [NEXT]` continues inside the procedure, as in QBasic.
- **Character set**: BASIC text is code page 437. `user/basic/basfont.h` (generated by `tools/gen_basfont.py` from Debian console-setup's VGA fonts) holds the $8 \times 8 / 14 / 16$ glyphs that `glyph ()` plots, and the Latin-1 $\leftrightarrow$ CP437 tables: the lexer translates string literals and `DATA`, the host translates keys, and the VM's `cstr () / pushL1 ()` translate what goes to or comes from the system (controls, paths, clipboard, dialogs); `craw ()` keeps screen text (`DRAWTEXT`) and `ENVIRON` as they are. A PC render of it: `run_games_test.sh BASICRT` (programs in `tools/tests/basic/rt/`).
- **The PC tools** (`pc/`, built on Linux by `sh pc/build.sh` into `pc/dist/`, which is committed): `obcore.dll` (`pc/obcore/obcore.cpp`, mingw-w64) is the core + `basscreen.h` with a Windows `ScreenHost` (`SD:/` mapped to a folder, `waveOut` sound, callbacks for the window, the picture, the controls and the dialogs) and a C API (`ob_check`, `ob_words`, `ob_run`, `ob_key / ob_keyheld / ob_mouse / ob_event / ob_stop`); `OnyxBasic.exe` (`pc/OnyxBasic`, .NET Framework 4.8 WinForms, `EnableWindowsTargeting`) is the editor (`EditorForm.cs`: modules split / composed like `qbasic`) and the runtime (`Runner.cs`: the VM on a worker thread, the page drawn scaled into a bitmap, WinForms controls). Wine + wine-mono can run both on Linux for a check.
- **Koton for Windows** (`pc/Koton`, built on Linux by `sh pc/Koton/build.sh` into `pc/dist/Koton/`, committed; `pc/build.sh` runs it too). Not a port: **the Onyx sources, unchanged** -- `user/Apps/koton`, `user/wtk`, FreeType, MeltySynth, the plugins `user/Apps/kp_*`, `user/bin/llm.cpp` + `mbedtls` -- built with MinGW-w64 over `pc/Koton/winkapi.cpp`, the kernel's ABI table on Win32: put at `KAPI_TABLE_VA` (`VirtualAlloc`) by a constructor that runs before all the others (`init_priority`), so `KT- $\rightarrow$ ...` works as on the Pi. What it gives: a Windows window whose client

area is the canvas (`get_chrome` answers "no frame": wtk draws none), the app's menu bar as a Windows menu (`set_menu`), its size as the work area (`win_geometry`: a maximised wtk `Root` follows the window -- `GUI_EVENT_DISPLAY_RESIZE` on `WM_SIZE`), the events queued by the window procedure and handed out by `pump_events` (the app's thread, as on Onyx); files with `SD:/` = the exe's folder (`ONYX_SD`, inherited by the programs it starts) and `C:/...` a Windows path, `.../main` and `SD:/bin/llm` being `main.exe`, `llm.exe`; threads, `wait_word` (`WaitOnAddress` re-checked every millisecond, as the Onyx kernel re-reads a sleeping word: a word another process changed wakes its sleeper); the sound ring played by WASAPI (shared mode, event driven; `winmm`, then a silent real-time drain, when not); `winmm` MIDI inputs as the USB MIDI events; processes in a job (they end with `Koton`), pipes for `spawn`; named services, mailboxes (a ring per process in a named mapping) and surfaces (named mappings) for the plugins; Winsock for `tcp_*` (the AI helper's HTTPS). `pc/Koton/onyxwin.h` (forced first: `-include`) stands for `onyxpp.hpp` (the C runtime's `new / delete`), `-DIMG_HOST_TEST` for the image codecs' libc stubs. Three things in the shared sources exist for it, none changing Onyx: `gui_value` (`kapi_abi.h`: an event's 64-bit value -- `long` on Onyx, `long long` where `long` has 32 bits) with the `GUI_PTR_*` macros in 64 bits, wtk's handlers declared with it; `kapi_memset/memcpy/memmove`'s weak definitions left out on `_WIN32` (PE has no weak symbols worth the name); wtk's file dialog lists `SD:` and the drives `C:...` on `_WIN32`. Checked under Wine (`Xvfb + wine explorer /desktop=onyx,1920x1080 ... \Koton.exe`, driven by `xdotool`): the demo song, playback, the editors, the plugins (a generator, an effect, an instrument: processes, editors, their sound), Compose with AI (`llm.exe` over HTTPS), the file dialog, closing. `pc/Koton/README.txt` is the user's page (copied into the folder).

- **Volume and Wi-Fi from the menu bar** (ABI v60): `user/volume.h` (`volume_save / volume_restore`: `SD:/etc/sound.ini`) for the menu bar's volume box and `/bin/volume` (`kapi_sound_volume` (`vol`, `mute`), `-1` keeps). The Wi-Fi menu is its own app, `Apps/wifimenu` (a borderless window: it takes the keyboard for the password, unlike the TOPMOST menu bar): `kapi_wlan_scan`, the known networks parsed from / written back to `SD:/etc/wpa_supplicant.conf` (several `network={}` blocks, `priority`), then `kapi_wlan_reconnect`; it closes when another window has the keys (`kapi_win_list`, `KAPI_WIN_KEYS`).
- **NintendoEMU** (`pc/NintendoEMU`): `nemucore.dll` (`core/nemucore.cpp`, `mingw-w64`) builds the emulator cores **unchanged** (`user/gb`, `gba`, `nes`, `snes`, `n64`, `gc`) behind a C API — `ne_open` (the system from the extension; a GameCube disc read on demand through `discRead`), `ne_set_keys` (the Windows key states) + `XInput` pad 0, mapped per system exactly as the Onyx apps do, `ne_run_frame` (`h`, `draw`), `ne_audio`, `ne_video` (the last picture, double-buffered under a lock: the game runs on its own thread), `ne_thumb` (the library's picture), `ne_save` (`<rom>.sav`, as on Onyx; a GameCube's is its **memory card** in slot A, a 2 MB raw image — Dolphin's `.raw` sizes accepted — inserted before the boot and written when `card[0].dirty`), `waveOut` (`ne_audio_*`; the GameCube's sound through `Machine::audioRead`). The N64 GFrames go to **OpenGL** (`ne_gl_attach` (`h`, `hwnd`) from the game's thread: a WGL context on the picture control, GLSL 1.20 — `colour = texel × colour + colour2`, alpha test, the batch's matrix, blending / depth / cull / wrap from the flags, as `kapi_gpu_render`; the machine's dirty textures uploaded; a CPU-drawn framebuffer as a textured quad), else `bas::swTriangles + g3raster` in bands of 16 rows on every core (`user/basic/bas3d.h`) at `ne_set_scale` × their size. A GameCube game with OpenGL 3.3 gets **core/gxgl.cpp** as its `Machine::gpu` (the GX's real pipeline on the GPU): the EFB is a framebuffer (RGBA8 + depth 24, `ne_set_scale` × 640 × 528, its rows the EFB's); a draw's vertices, the XF memory and the `GxState` go to three ring buffers (persistently mapped, their quarters fenced; else unsynchronized maps) and two uniform blocks; the vertex shader transforms, lights (the two channels, 8 lights: spot / specular attenuation, diffuse functions) and makes the texture coordinates (texgens, dual texture); the fragment shader is the TEV in integers as the hardware

(the lerp with $c + c >> 7$, the scale inside it, compare modes, swap tables, konst colours, the last stage into PREV, & 255), indirect texturing (matrices, wraps, bump alpha), the alpha test, the z texture, the fog (with its range adjustment), RGBA6 / RGB565 EFB formats, the destination alpha (dual-source blending); GX blending / logic ops / depth / culling / scissor are GL state. Each TEV configuration gets a **specialized program** (the ubershader's code with the stages unrolled and the state's choices as constants — $\sim 1.6\times$ faster on an integrated GPU), compiled by the driver's threads when it has `KHR_parallel_shader_compile` (the ubershader meanwhile), else at once (the driver caches them on disk: a pause only the first time a configuration is seen). Consecutive draws of one state are one draw call; textures are uploaded with their mipmaps, sampled through cached sampler objects. An EFB copy to a texture renders the rectangle into a texture of ours converted as the texture decoder would read it (intensity $Y = 0.257 R + 0.504 G + 0.098 B + 16$, RGB565, RGB5A3, the depth's bytes for Z formats), box-filtered when halved; a copy to the XFB is the picture, presented into the window (`gxgl_present`). `NEMU_GX_UBER=1` forces the ubershader, `NEMU_GX_SYNC=1` compiles at once, `NEMU_GX_DUMP=<prefix>` writes the GPU's picture every 250 fields (+ what GL is). Pads: XInput, else the first Windows joystick (winmm `joyGetPosEx`) through `ne_pad_map` (PAD_* bit $\rightarrow$ button / axis end / hat direction; `PadDialog.cs`). `NintendoEMU.exe` (.NET 4.8 WinForms): `MainForm` (the library, `library.txt`, the pictures made by a worker thread), `GameForm` (the game thread: paced by the sound queue — ~ 70 ms — when the game makes sound, else by the clock, a picture skipped to catch up; the window draws with `StretchDIBits`). View $\triangleright$ 3D Renderer $\triangleright$ Use the High-Performance Graphics Card (`GpuPreference` in `GameForm.cs`) writes Windows' own per-application choice (HKCU\Software\Microsoft\DirectX\UserGpuPreferences, what Settings $\triangleright$ Display $\triangleright$ Graphics writes) for a laptop with two graphics cards: a .NET program cannot export `NvOptimusEnablement`. Test: `sh tools/tests/run_nemu_test.sh` (the C API on Linux: each core a few frames, a picture, the library pictures; `NEMU_ROMS=<folder>` of ROMs).

- **Tests:** `sh tools/tests/run_basic_test.sh` builds the core with a console host (`tools/tests/basic/host_main.cpp`, graphics / controls logged as text) under ASan + UBSan and compares `tools/tests/basic/progs/*.bas` with their `.out` (`--update` rewrites them).

10. Extending the kapi ABI

The ABI is **append-only**. To add a kernel function callable by apps, there are **5 points** to touch (all in the same direction, at the end):

1. `kernel/include/kern/kapi_abi.h` — add the function pointer **at the end** of struct `TKApiTable` (never in the middle, never reorder), with a version comment, and **increment** `KAPI_ABI_VERSION`.

```
// --- v22 additions --- (next free version; v21 added the TCP sockets)
int (*ma_fonction) (int arg);
```

2. `kernel/sys/kapi.cpp` — implement `extern "C" int kapi_ma_fonction(int arg)`. It runs in the app's context (use `CurrentAS()` for the current address space if needed).
3. `kernel/sys/kapitable.cpp` — declare the extern "C" prototype and **assign the pointer** in `KApiTableInit()`: `t->ma_fonction = kapi_ma_fonction;`
4. `user/kapi.h` — add the inline wrapper:

```
static inline int kapi_ma_fonction (int arg) { return KT->ma_fonction (arg); }
```

5. Rebuild (`make`). The apps that want the new function use it; the old ones keep working (they ignore the new field).

Golden rule: never change the signature or the order of an existing field. If some semantics must change, add a **new** entry. An app can query `((const struct TKApiTable *)KAPI_TABLE_VA)->version` to find out what is available.

If you add a new **GUI event** or a **window flag**, keep the values synchronized between `kernel/gui/window.h` and the `#defines` in `user/kapi.h` (commented "must match").

11. Coding conventions

- **Kernel (C++):** Circle style. `CXxx` classes, `m_Xxx` members, CamelCase methods, `boolean/TRUE/FALSE` and Circle's `u8/u16/u32/u64` types. No exceptions or RTTI. `new/delete` go through Circle's heap.
- **Userland (C):** freestanding C. `ax_` prefix for the `applib.h` helpers. Globals `g_XXX`. Bounded static buffers (no dynamic allocation on the app side in general).
- **kapi:** `extern "C"` functions named `kapi_XXX` on the kernel side; inline wrappers `kapi_XXX` on the app side.
- Respect the **comment density** and the **idiom** of the file you are modifying.
- **Git:** commit into the **Onyx repo** explicitly — the current working directory (cwd) drifts; a bare `git` may land in the wrong repo. (Note: `circle/` is not committed in this repo.)

12. Debugging on hardware

Bring-up is done **directly on the Pi 4** (no QEMU raspi4b). Tools:

- **On-screen exception dump:** an EL1 synchronous fault (or an EL0 fault) paints a panic + register dump on the HDMI framebuffer (`PanicToScreen` + Circle's handler). Note the `ELR` (faulting PC).
- **addr2line:** `aarch64-none-elf-addr2line -e kernel8-rpi4.elf <ELR>` to locate the faulting line (keep the unstripped `.elf` next to the `.img`).
- **Remote shell:** `/bin/telnetd` (autostarted, TCP port 23) serves the `cmd` shell over the network; from the dev machine, `python tools/onyx-telnet.py <pi-ip>` (or any telnet client) — handy to run `kmsg`, `ps`, `kill`, tests, without the Pi's keyboard. Its server side uses the ABI v37 `kapi_tcp_listen/kapi_tcp_accept`.
- **Remote desktop:** `/bin/vncd` (autostarted, VNC port 5900) — watch and drive the GUI from any VNC viewer, no monitor needed. It grabs the screen with `kapi_screen_grab` and injects input with `kapi_inject_pointer/kapi_inject_key` (ABI v38); it is a newlib program linked with the vendored `zlib` (`ZLIB_PROGS` in `user/bin/Makefile`).
- **Window-level remote desktop:** `/bin/rdpd` (autostarted, port 3390) + the Windows client `pc/OnyxRemote` (.NET Framework 4.8, built by `sh pc/build.sh` into `pc/dist/OnyxRemote.exe`): one MDI window (+ `TelnetForm`: a telnet console on `telnetd`, its own window -- IAC dropped, `ESC [2J` clears, the line sent whole + CR LF, `telnetd` echoes it) -- the Onyx menu bar across its top (stretched: `BarView`, its drop-down menus in a colour-keyed layer over the children), each Onyx window a child window (native frame or the Onyx one), the desktop + bubbles as the MDI area's background -- composited by the PC: nothing is composited for it on the Pi. `rdpd` lists the windows (ABI v56 `kapi_win_list`), reads only those whose `gen` changed (`kapi_win_read`), compares their 64 × 64 tiles with what the client has and sends the changed runs, LZ4-compressed (its own compressor, the standard block format), 32 or 16 bits a pixel; a frame when

`chromeGen` changes; the desktop (`KAPI_WIN_DESKTOP`, the wallpaper + the backmost windows) only when the client asks for it (message 7). The hello carries the kernel's kapi version (the client warns below 56). The client sends the pointer in window coordinates (rdpd adds the window's place, raises it when clicked), keysyms (`user/bin/remotekeys.h`, shared with `vncd`: specials, modifiers, letters / digits as held keys only) and the characters typed (the PC's layout). The protocol is described at the top of `user/bin/rdpd.c`. Host test (mock kapi, a Python client): `sh tools/tests/run_rdpd_test.sh`.

- **Serial console:** `config.txt` must have `enable_uart=1` (PL011 clock). The boot log goes **also** to the HDMI screen (`CScreenDevice`) so it is readable without a serial cable.
- **Post-mortem console:** on an app's exit, the compositor clears and the logger is shown on the framebuffer (see [Kernel internals §11](#)).

13. Known pitfalls

- **kapi_resize_window keeps the buffer's size and row pitch.** The window buffer is made once, at the size given to `kapi_create_window`; resizing changes the size shown (clamped to that buffer), not the buffer, and the compositor reads its rows with the **creation width** as pitch. Draw with that pitch: `canvas.adopt (kapi_resize_window (w, h), w, h, creationWidth)`. To offer several sizes, create the window at the largest one (the emulators: their 4x zoom). Adopting with pitch = the new width gave a doubled, interlaced picture in `gbemu` at zoom 2x. The window's **frame follows the new size**: call `gtk::wk_decorate_window ()` after the resize so the title bar and borders are redrawn at it (else the frame keeps the old drawing).
- **kapi_mkdir / kapi_remove / kapi_rename return 0 on success** (-1 on failure), like POSIX — not a boolean. Test `== 0` for success (a `!kapi_rename (...)` "failure" check silently treated every successful move as failed; fixed in `trash.h`, `fsutil.h`, the File Viewer and `ftpd`).
- **Host tests** (`tools/tests/`): `run_fs_test.sh` (the Circle fork's FatFs + `diskio.cpp` sector cache on a RAM disk: 4000 random file operations checked against a model, cache on and off, same disk image — run it after touching `circle/addon/fatfs`), `run_trash_test.sh` (`trash.h` + `fsutil.h` against a mock kapi with the kernel's return conventions) and `run_ftpd_test.sh` (ftpd over real sockets, driven by Python's `ftplib`: login, LIST/NLIST, RETR/STOR round trip, MKD/RMD, RNFR/RNTO, DELE, root jail, PORT, two concurrent sessions) and `run_ftpfs_test.sh` (the ftpfs FTP client against `pyftplib` — `pip install pyftplib` — and against ftpd: LIST/MLSD, RETR, a 300 KB STOR round trip, MKD, RNFR/RNTO, DELE/RMD, a missing file, reconnecting after a dropped control link). The socket mock reproduces Circle's "one segment per receive, the rest is dropped". Run them after touching those files.
- **Hardware float is opt-in.** Apps are integer-only by default (`-mgeneral-regs-only`); the kernel now saves the full FP/SIMD state on every trap, so an app may opt into `float/double` by building without `-mgeneral-regs-only` and with `-mcpu=cortex-a72` (see §5). Preemption saves the full FP state too (the preemption trampoline stores it before yielding); keep it that way if the preemption path changes — the cooperative `TaskSwitch` only keeps `d8-d15`.
- **L3 tables shared with the kernel.** On the kernel side, never free an L3 table from the user area without checking that it is not shared with the kernel's L2 (cf. [Kernel internals §4](#)). Otherwise: global corruption.
- **Do not free the ABI table page.** It is global to the kernel; the destruction of an address space already skips it.

- **DEPTH=32 for Circle.** `GImage` renders 32-bit; forgetting `-d DEPTH=32` (or changing `DEPTH` without `make clean` in `circle/lib`) gives wrong colors/breakage.
- **The kernel is not preemptive.** Apps are preempted, but kernel code (a `kapi_*` call, a kernel task) is not: a long kernel loop that never yields still stops every other task. Put `if (IsReschedPending()) Yield ();` in any such loop (the stall watchdog's `stall:` log lines point at them).
- **Threads and the kernel** (v67): a `kapi` that waits (`Yield`, an event) may now find another task of the **same** process inside the kernel when it resumes — per-process state touched across a wait (the window's event queue, a dialog, a file handle) is no longer the caller's alone. And `CScheduler::EnumerateTasks`'s callback must not yield (the task list is a linked list the reaper frees nodes from). An app's tasks are one group (`TerminateTask` on one ends them all; `TerminateGroup`); list the process once (`pAS->GetMainTask ()`).- **Circle LF renormalization.** On Windows, Circle is checked out in CRLF; renormalize once (cf. §2) otherwise the build breaks.
- **The right Circle.** Patch `Zircon/circle`, not another clone.